

ThreatNeXus

Technical Delivery, Deployment and Operations Dossier

Developed during an internship with PKCERT/NCERT

Research prototype — not production-approved

Document ID	TNX-DOC-9C-DOSSIER-001
Version	1.0
Date	2026-08-10
Repository commit	c3f8a4b
Authoring branch	docs/phase-9c-pkcert-technical-dossier

ThreatNeXus — Technical Delivery, Deployment and Operations Dossier

Part 1 — Document control

1.1 Purpose and audience

This dossier is the single, consolidated technical reference for ThreatNeXus, written for a PKCERT/NCERT technical reviewer, an internship supervisor, or an engineer picking up the codebase for the first time. It explains what the system is, why it was built, how it works today, how to install and operate it, and — separately and explicitly — what a production deployment would still require. It supersedes no individual document under `docs/`; it reconciles and consolidates them into one printable reference.

1.2 Status statement

ThreatNeXus is a defensive cybersecurity research prototype, developed during a student internship with PKCERT/NCERT. It is not an official PKCERT/NCERT production platform. No national deployment is claimed or implied. It has not been independently security-audited, has no high-availability or disaster-recovery story, ships with local-development defaults, and has never processed real constituent data. Nothing in this document should be read as a certification, endorsement, or production-approval by PKCERT/NCERT.

1.3 Document identification

Field	Value
Document ID	TNX-DOC-9C-DOSSIER-001
Title	ThreatNeXus — Technical Delivery, Deployment and Operations Dossier
Version	1.0
Date	2026-08-10
Repository	github.com/haiderchattha99-ali/ThreatNeXus
Source commit	c3f8a4b (main, PR #18 — Phase 9B.1 merged)
Authoring branch	docs/phase-9c-pkcert-technical-dossier
Prepared by	ThreatNeXus internship team (M. Ismail, Ali Haider, Aun Zulfiqar, Eshaal Khan); drafting assistance by an AI development tool, human-directed and human-reviewed before publication
Reviewed by	Not yet reviewed by PKCERT/NCERT — field left unsigned; no review record exists to cite
Approved by	Not yet approved — field left unsigned; this document carries no approval status
Distribution	Internal and PKCERT/NCERT review use. Not for public distribution. Contains no secrets and no real constituent data (see §1.6).
Classification	Unclassified — internship deliverable

No signature, approval date, or classification beyond the above is asserted anywhere in this document. Where a fact could not be verified against the running codebase, it is labeled unknown, not benchmarked, or proposed rather than estimated.

1.4 Revision history

Version	Date	Author	Change
0.1	2026-08-10	ThreatNeXus internship team (drafting assistance: AI development tool)	Initial consolidated draft, reconciling docs/*.md (Phase 9A/9B.1 baseline) against the running codebase at c3f8a4b
1.0	2026-08-10	ThreatNeXus internship team	First complete draft: all 13 parts, diagrams, appendices, PDF visual QA pass

1.5 How this document was produced

Every factual claim in this dossier was checked against one of three sources, in this order of precedence: (1) the running source code at commit c3f8a4b — route files, Prisma schema and migrations, backend/.env.example, docker-compose*.yaml, .github/workflows/ci.yaml; (2) the existing docs/*.md package (Phase 9A/9B.1), itself previously reconciled against the code; (3) docs/ai/SECURITY.md, which records phase-by-phase implementation evidence. Where these disagreed, the code won. Where a number (a test count, a migration count) could not be freshly re-executed in this drafting session, it is labeled with the commit it was last verified at rather than presented as freshly measured — see §11 and the Testing appendix for the specific reconciliation performed.

1.6 Distribution and sensitivity statement

This document contains no API keys, no passwords, no JWT secrets, and no real constituent or victim data. Every credential-shaped value shown anywhere in this dossier is an environment-variable **name**, not a value, or an explicitly labeled placeholder (e.g. <your-strong-local-password>). backend/.env (the real, gitignored environment file on the development machine) was never opened, read, printed, or referenced while producing this document — only backend/.env.example, which contains placeholders only, was consulted. See Part 7 and the Security appendix for the full secret-handling policy.

1.7 Table of contents

Part 1 — Document control

- 1.1 Purpose and audience
- 1.2 Status statement
- 1.3 Document identification
- 1.4 Revision history
- 1.5 How this document was produced
- 1.6 Distribution and sensitivity statement
- 1.7 Table of contents
- 1.8 Abbreviations and glossary

Part 2 — Executive and product overview

- 2.1 The problem
- 2.2 What ThreatNeXus is
- 2.3 Why it was built this way
- 2.4 Intended users

- 2.5 Value delivered
- 2.6 In-scope capabilities (implemented and verified)
- 2.7 Explicitly out of scope
- 2.8 End-to-end workflow (summary; detailed in Part 4 and Part 5)
- 2.9 Honest limitations (summary; full list in the Appendices)
- 2.10 Phase history
- 2.11 Current project status

Part 3 — Software requirements specification

- 3.1 Scope and conventions
- 3.2 Functional requirements
- 3.3 Non-functional requirements
- 3.4 Security requirements
- 3.5 Audit requirements
- 3.6 Role and capability requirements
- 3.7 Requirement families covering ingestion, provider evidence, and AI (cross-references)
- 3.8 Requirements traceability

Part 4 — Technical architecture

- 4.1 System context
- 4.2 Containers and components
- 4.3 Backend module map
- 4.4 Main data flow
- 4.5 Provider adapter pattern
- 4.6 Authentication and authorization
- 4.7 Audit flow
- 4.8 AI assistance flow
- 4.9 Trust boundaries
- 4.10 Current Docker topology
- 4.11 Proposed production topology (see Part 9 — not implemented)
- 4.12 Technology choices and why

Part 5 — Data model and semantics

- 5.1 Main entities and relationships
- 5.2 Finding lifecycle
- 5.3 Risk snapshots and factor contributions
- 5.4 Cases, reviews, and separation of duties
- 5.5 Notifications and immutable revisions
- 5.6 Audit logs
- 5.7 Framework mappings
- 5.8 Provider-specific enrichment rows
- 5.9 AI suggestion records
- 5.10 Status vocabularies: `UNAVAILABLE`, `RESTRICTED`, stale and absent
- 5.11 Export versus delivery

Part 6 — External provider guide

- 6.1 Shared behavior across all six providers
- 6.2 AbuseIPDB — IP reputation (the required first provider)
- 6.3 NVD, CISA KEV, FIRST EPSS — vulnerability metadata
- 6.4 Censys — internet exposure / attack surface
- 6.5 GreyNoise — internet noise / scanning context

- 6.6 Shodan — exposed service / banner / port intelligence
- 6.7 Netlas — cross-source attack-surface / DNS / certificate intelligence
- 6.8 Providers not integrated
- 6.9 Quota protection

Part 7 — AI governance

- 7.1 The one operator switch
- 7.2 No live AI provider ships
- 7.3 The two AI surfaces
- 7.4 Why acceptance can never exceed manual authority
- 7.5 What AI is structurally unable to do
- 7.6 AI output is untrusted input
- 7.7 Staleness
- 7.8 Prompt-injection handling
- 7.9 Human review and separation of duties
- 7.10 Frontend behavior
- 7.11 Actions AI is structurally unable to perform (consolidated)
- 7.12 Limitations, stated honestly

Part 8 — Current installation and deployment manual

- 8.1 Supported use
- 8.2 Required software
- 8.3 Repository setup (Windows / PowerShell — the primary presentation machine)
- 8.4 Environment-variable categories
- 8.5 JWT secret generation
- 8.6 Starting the stack
- 8.7 Optional provider configuration
- 8.8 Database creation and Prisma migrations
- 8.9 Seed users and demo seed
- 8.10 Online mode, offline rehearsal mode, live-provider mode
- 8.11 Health checks
- 8.12 Startup, shutdown, reset
- 8.13 Backup and restore
- 8.14 Troubleshooting
- 8.15 What must never be committed

Part 9 — Proposed production deployment blueprint

- 9.1 Production network topology (proposed)
- 9.2 Gap matrix
- 9.3 Production acceptance criteria (proposed, not yet met)

Part 10 — Operations and maintenance

- 10.1 Startup / shutdown SOP
- 10.2 Service health and logs
- 10.3 Database and migration checks
- 10.4 Provider status checks
- 10.5 Rate-limit events
- 10.6 Key expiry and provider failures
- 10.7 Backup and restoration
- 10.8 Offline rehearsal
- 10.9 Common failures and recovery

- 10.10 Maintenance windows
- 10.11 Release checklist (current state)
- 10.12 Rollback checklist (current state)
- 10.13 Demo-day operational checklist

Part 11 — Testing, CI and security assurance

- 11.1 Four independent verification layers
- 11.2 Unit and integration tests
- 11.3 Evaluators
- 11.4 Frontend and Chromium browser suite
- 11.5 CI pipeline
- 11.6 Provider no-live-call guarantee
- 11.7 Known flake policy
- 11.8 Security controls (summary; full detail in Part 3 §3.4 and the Security appendix)
- 11.9 Current security-assessment limitation

Part 12 — User and administrator guide

- 12.1 Signing in
- 12.2 Dashboard
- 12.3 Findings
- 12.4 Upload (ADMIN, ANALYST)
- 12.5 Triage (ADMIN, ANALYST)
- 12.6 Cases
- 12.7 Framework mapping
- 12.8 AI assistance
- 12.9 Notifications (ADMIN/ANALYST draft; ADMIN/REVIEWER approve — never VIEWER)
- 12.10 Denied and restricted states
- 12.11 Provider configuration (ADMIN)
- 12.12 Admin limitations (stated plainly)

Part 13 — Appendices

- 13.1 Full capability matrix
- 13.2 Current API route catalogue
- 13.3 Environment-variable catalogue
- 13.4 Provider comparison
- 13.5 Status and error vocabulary
- 13.6 Requirements traceability matrix (representative sample)
- 13.7 Production-readiness checklist (starting point, not yet completed)
- 13.8 Known limitations (consolidated)
- 13.9 Document references

(This list is generated at build time from the document's own Part/section headings, with clickable links and matching PDF bookmarks — the placeholder above is replaced by the build script.)

1.8 Abbreviations and glossary

Term	Meaning
CERT / NCERT	Computer Emergency Response Team / National CERT
PKCERT	Pakistan CERT — the internship host organization referenced throughout this document
IOC	Indicator of Compromise (here: an IPv4 address)
CVE	Common Vulnerabilities and Exposures identifier
KEV	CISA Known Exploited Vulnerabilities catalogue
EPSS	FIRST Exploit Prediction Scoring System
ATT&CK	MITRE ATT&CK adversary-behavior framework
CSF	NIST Cybersecurity Framework (2.0)
CIS	CIS Controls (v8)
RBAC	Role-based access control (here: capability-based, non-hierarchical)
JWT	JSON Web Token, this system's bearer-authentication credential
TLP	Traffic Light Protocol sensitivity labeling
Finding	The system's dedup unit: one row per (indicator_value, port, protocol, report_type)
Case	An analyst-managed, organization-bound container for one or more Findings
Dedup key	(indicator_value, port, protocol, report_type) — the tuple that identifies "the same exposure"
Persistence	The same open Finding observed again — bumps an occurrence count, no new row
Recurrence	A closed Finding's case reopened because the exposure was seen again
Remediation proxy	An organization's <i>statement</i> that something was fixed — never independently verified
Export ≠ delivery	Producing a notification file and actually sending it are separate, separately recorded events
RPO / RTO	Recovery Point Objective / Recovery Time Objective — stakeholder decisions, not yet set (Part 9)

Part 2 — Executive and product overview

2.1 The problem

A national CERT receives recurring third-party exposure reports — in this project's case, modeled on Shadowserver-style CSV feeds — that list hosts with a specific network-level exposure (for example, an Internet-reachable RDP service). Handled by hand, a spreadsheet-based process struggles with three specific failures: the same exposure gets re-reported and re-triaged as if new every time it reappears; nobody can say with confidence whether a previously-flagged host was actually fixed or simply dropped off one report by chance; and notifications to the affected organization go out without a consistent, auditable trail of who approved what, when.

2.2 What ThreatNeXus is

ThreatNeXus is a defensible CERT triage and constituent-notification workflow. It ingests a Shadowserver-style exposure report, deduplicates it into persistent Findings, attributes each Finding to an owning constituent organization, enriches and scores it deterministically, carries it through an analyst case with independent review, drafts and — after human approval — exports a notification to the affected organization, and records what came back. A complete audit trail sits behind every step.

2.3 Why it was built this way

Three design commitments recur throughout this document and are treated as load-bearing, not stylistic:

- **Export is not delivery.** Producing an approved notification file is one event; a human actually sending it, and whatever the organization does in response, are separate events this system records but does not perform or verify.
- **A remediation proxy is not verified remediation.** When an organization states that an issue was fixed, the system records that they said so — never that it independently confirmed it.
- **Provider evidence is supporting context, never proof.** A reputation score, an exposure lookup, or an EPSS percentile informs an analyst's decision; none of them decides anything by itself.

2.4 Intended users

Role	Who they are in practice	What they do in the system
ADMIN	A system administrator / lead	Every capability, including two explicit self-approval override grants and user/system administration
ANALYST	A CERT analyst doing triage	Ingests reports, triages Findings, builds cases, drafts notifications, requests enrichment/AI assistance
REVIEWER	An independent second analyst	Approves case closures and notifications; approves/rejects Finding-level AI drafts — never does ANALYST work

Role	Who they are in practice	What they do in the system
VIEWER	Oversight / management	Read-only; deliberately excluded from notification content, which is narrower than the general threat feed

2.5 Value delivered

- Deduplicated, persistent Findings with recurrence detection, so the same exposure is never silently double-counted or silently forgotten.
- A deterministic, explainable risk score whose every factor contribution is stored and can be shown to an auditor, never a black-box number.
- An auditable case workflow with structural separation of duties (the requester of a closure or notification can never also approve it).
- Constituent notifications a named human approves before anything is exported, with immutable revision history so an edit after approval provably invalidates that approval.
- Six live, optional third-party intelligence providers behind one abstraction, each failing safely without ever blocking the core workflow.
- Two independent AI-assistance surfaces that are structurally unable to make a final decision, off by default, with no live AI provider shipped.

2.6 In-scope capabilities (implemented and verified)

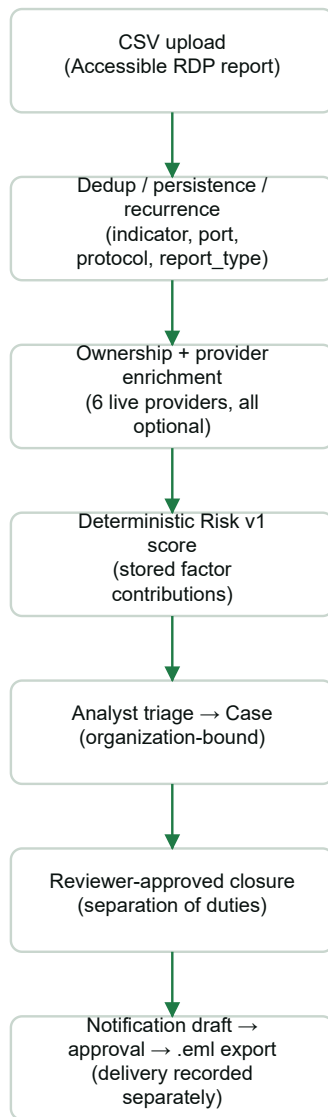
- Ingestion of one Shadowserver-style report type — **Accessible RDP exposure** — via CSV upload, carried end to end to case closure.
- Deduplication, persistence, and recurrence on the key (`indicator_value`, `port`, `protocol`, `report_type`).
- Ownership resolution (exact IP → longest-prefix CIDR → ASN) with analyst override.
- Six live intelligence providers: AbuseIPDB (IOC reputation), NVD/CISA KEV/FIRST EPSS (vulnerability metadata), Censys, Shodan, Netlas (internet exposure / attack surface), GreyNoise (internet-noise reputation) — all optional, all fail-safe.
- Deterministic, explainable Risk v1 scoring with stored per-factor contributions.
- Analyst case workflow: triage → case → evidence → reviewer-approved closure → recurrence reopening.
- Manual framework mapping (MITRE ATT&CK, NIST CSF 2.0, CIS Controls v8) with a server-enforced ATT&CK observed-behavior evidence rule.
- Two independent, disabled-by-default AI-assistance surfaces (case-level mapping suggestions, Finding-level narrative drafts), both human-approved only.
- Notification drafting, immutable revisions, reviewer approval bound to an exact revision, approved-only manual `.eml` export, and separately recorded delivery tracking.
- A truthful analyst dashboard in which an unknown or restricted figure is never rendered as a real zero.
- Docker Compose local/demo deployment, a CI pipeline, and a deterministic demonstration dataset.

2.7 Explicitly out of scope

Live scheduled Shadowserver API ingestion · automatic notification sending (no SMTP/webhook client exists anywhere in the codebase, not even a disabled one) · automatic remediation verification · SIEM/EDR

integration · threat-actor attribution · automatic compliance assessment · a production deployment target (Part 9 describes what one would require, explicitly as a proposal, not as a claim).

2.8 End-to-end workflow (summary; detailed in Part 4 and Part 5)



End-to-end workflow (summary)

2.9 Honest limitations (summary; full list in the Appendices)

- Single report type (Accessible RDP); manual CSV ingestion only, no live Shadowserver API pull.
- No live AI provider ships; the AI contract, a disabled provider, and a test-only mock exist.
- **Finding closure has no production write path.** The recurrence/reopen engine is real and is proven by `eval:phase1/eval:phase3`, but nothing reachable through the current UI or API produces the closed state it reopens from — this is stated here rather than staged to look otherwise.
- No production deployment target exists. Docker Compose is a developer/demonstration runtime only.
- No in-app user management; accounts come from a seed script or a direct database change.

- Rate limiting is in-process, correct for a single-process prototype, wrong for a horizontally scaled deployment.

2.10 Phase history

Phase	Delivered
0	Foundation: config validation, audit logging, RBAC on every protected route, hardened auth, Vitest + Supertest, local PostgreSQL via Docker
1	Shadowserver-style ingestion → Finding lifecycle (dedup / persistence / recurrence)
2	Ownership resolution, AbuseIPDB IOC enrichment, NVD/KEV/EPSS vulnerability enrichment, deterministic Risk v1
3	Analyst workflow — cases, evidence, reviewer-approved closure, separation of duties
4	Notification workflow — drafting, immutable revisions, review, .eml export, delivery tracking
5	Framework mapping (manual first) + guarded, disabled-by-default AI mapping assistance
6 (6.1–6.3)	Analyst frontend, truthful dashboard integrity model, motion design, pinned ATT&CK catalogue
7	Evaluation harness completion, rate limiting, release-candidate hardening
8 / 8B–8F	Provider-evidence gap-fill, then Censys, Finding-level AI assistance, GreyNoise, Shodan, Netlas
9A	Professional delivery documentation package (docs/*.md)
9B / 9B.1	PKCERT presentation deck and demo assets
9C	This consolidated technical dossier

2.11 Current project status

As of commit `c3f8a4b` (branch `main`, PR #18 merged): all phases through 9B.1 are delivered and CI-green on `main`. This dossier (Phase 9C) is documentation and packaging only — it adds no product feature and changes no application behavior. No PR is opened for this phase and `main` is not touched; see the Document Build Notes for the exact validation performed.

Part 3 — Software requirements specification

3.1 Scope and conventions

This SRS section describes requirements **as they exist in the running system today**, plus a small, clearly separated set of requirements a production deployment would additionally need (cross-referenced to Part 9, never mixed into the "implemented" list). Every requirement below is one of exactly two states:

- **Implemented** — verified against the code at commit `c3f8a4b`.
- **Production requirement** — not implemented in this prototype; tracked in Part 9's gap matrix.

No requirement in this section describes something the implementation does not actually satisfy. Identifiers are stable and namespaced: TNX-FR-* (functional), TNX-NFR-* (non-functional), TNX-SEC-* (security), TNX-AUD-* (audit), TNX-AI-* (AI-assistance constraints).

3.2 Functional requirements

ID	Requirement	Status
TNX-FR-001	The system shall accept a Shadowserver-style Accessible-RDP CSV upload from an ADMIN or ANALYST caller	Implemented
TNX-FR-002	The system shall validate CSV structure and each row independently, rejecting structurally invalid rows without failing the entire report	Implemented
TNX-FR-003	The system shall deduplicate ingested rows on (indicator_value, port, protocol, report_type)	Implemented
TNX-FR-004	An existing open Finding observed again shall be treated as persistence: its occurrence count increments; no new Finding row is created	Implemented
TNX-FR-005	An existing closed Finding observed again shall be treated as recurrence: the linked Case reopens and the reopening is audited	Implemented
TNX-FR-006	The system shall resolve Finding ownership by exact IP match, then longest-prefix CIDR match, then ASN, with an analyst override capability	Implemented
TNX-FR-007	The system shall enrich a Finding's IOC reputation automatically on ingestion via a provider abstraction (AbuseIPDB or a mock), never blocking ingestion on provider failure	Implemented
TNX-FR-008	The system shall support analyst-triggered, on-demand exposure enrichment (Censys, Shodan, Netlas) and reputation enrichment (GreyNoise) per Finding	Implemented

ID	Requirement	Status
TNX-FR-009	The system shall support analyst-asserted CVE association per Finding, enriched with NVD metadata, CISA KEV status, and FIRST EPSS score	Implemented
TNX-FR-010	The system shall compute a deterministic Risk v1 score per Finding, storing every factor's contribution and applicability (APPLIED/NOT_AVAILABLE/NOT_APPLICABLE)	Implemented
TNX-FR-011	The system shall allow an ADMIN or ANALYST to triage a Finding and open an organization-bound Case	Implemented
TNX-FR-012	A Case shall link one or more Findings as evidence via a join table, never a single foreign key	Implemented
TNX-FR-013	The system shall record organization responses against a Case, including a REMEDIATED response state	Implemented
TNX-FR-014	Case closure shall require reviewer approval from a person other than the requester, except via an explicit ADMIN override capability	Implemented
TNX-FR-015	A REMEDIATED closure request shall be refused unless a REMEDIATED organization response is already recorded	Implemented
TNX-FR-016	The system shall support manual MITRE ATT&CK, NIST CSF 2.0, and CIS Controls v8 mappings against a Case, append-only	Implemented
TNX-FR-017	An ATT&CK mapping shall be refused server-side unless its evidence basis is OBSERVED_BEHAVIOR, its rationale passes a substance floor, its evidence links to the case or a linked finding, and a lexical guard confirms the rationale describes behavior rather than restating exposure	Implemented
TNX-FR-018	The system shall support two independent, disabled-by-default AI-assistance surfaces: case-level framework-mapping suggestions and Finding-level summary/explanation drafts	Implemented
TNX-FR-019	An AI suggestion shall remain inert data until a named human reviewer accepts or rejects it	Implemented
TNX-FR-020	Accepting an AI mapping suggestion shall promote it through the identical write path a manual mapping uses	Implemented
TNX-FR-021	The system shall support notification drafting from Case evidence, with each edit producing a new immutable revision	Implemented
TNX-FR-022	Editing an approved notification revision shall invalidate that approval by stored state	Implemented

ID	Requirement	Status
TNX-FR-023	Notification review approval shall bind to the exact revision reviewed; a later edit shall never inherit that approval	Implemented
TNX-FR-024	The export endpoint shall refuse to export any notification whose status is not Approved or whose approved_by is null	Implemented
TNX-FR-025	An approved notification shall export as an RFC 5322 .eml file for manual sending; the system shall never send it itself	Implemented
TNX-FR-026	Notification delivery shall be recorded as a separate, explicitly human-observed fact, never inferred from the export event	Implemented
TNX-FR-027	The dashboard shall present every figure as a { value, availability, source, asOf } tuple, never a bare number	Implemented
TNX-FR-028	A dashboard figure the caller's role may not read shall render as RESTRICTED, never as zero	Implemented
TNX-FR-029	A dashboard figure that could not be computed shall render as UNAVAILABLE, never as zero	Implemented
TNX-FR-030	Public self-registration shall always create a VIEWER account regardless of the requested role	Implemented
TNX-FR-031	The system shall provide a live scheduled Shadowserver API ingestion path	Production requirement (Part 9) — not implemented; out of scope for this delivery
TNX-FR-032	The system shall provide an in-app path to reach a Finding's closed state through the UI, so recurrence can be demonstrated end to end without driving the evaluators directly	Production requirement (Part 9) — not implemented (§2.9)
TNX-FR-033	The system shall provide in-app user management (create, promote, disable an account)	Production requirement (Part 9) — not implemented

3.3 Non-functional requirements

ID	Requirement	Status
TNX-NFR-001	Every core workflow (ingestion through notification export) shall complete correctly with AI_ENABLED=false, the shipped default	Implemented

ID	Requirement	Status
TNX-NFR-002	Every core workflow shall complete correctly with zero provider API keys configured	Implemented
TNX-NFR-003	The application shall refuse to start without a JWT_SECRET of at least 32 characters that is not a recognizable placeholder	Implemented
TNX-NFR-004	A provider failure (timeout, rate limit, invalid key, malformed response) shall never block ingestion or the workflow step it is attached to	Implemented
TNX-NFR-005	All Prisma migrations shall be additive; no migration shall alter or drop an existing column	Implemented (23/23 migrations, verified by CI's frozen migration-history check)
TNX-NFR-006	The system shall apply three independent rate-limit buckets (auth, upload, provider execution), each independently configurable	Implemented
TNX-NFR-007	The system shall support a fully offline rehearsal mode in which every external provider host is unreachable, with identical workflow behavior to the online mode	Implemented (docker-compose.offline.yml)
TNX-NFR-008	The system shall run its full backend and evaluator suite without ever making a live network call to a third-party provider	Implemented — enforced at three independent layers (§11.6)
TNX-NFR-009	The frontend shall honor prefers-reduced-motion for all non-essential animation and remain fully usable with motion disabled	Implemented
TNX-NFR-010	Rate limiting shall operate correctly under a single Node process; horizontal scaling shall require a shared store	Implemented for single-process; horizontal scaling is a production requirement (Part 9)
TNX-NFR-011	The system shall provide documented, benchmarked hardware sizing for a production deployment	Not benchmarked — no formal load test has been performed against this codebase; Part 8 provides demonstration-workstation guidance only, explicitly labeled as such
TNX-NFR-012	The system shall provide horizontal scalability for the backend API	Production requirement (Part 9) — not implemented; current deployment is single-instance

3.4 Security requirements

ID	Requirement	Status
TNX-SEC-001	The backend shall be the sole authorization boundary; frontend capability checks shall be presentation convenience only	Implemented

ID	Requirement	Status
TNX-SEC-002	Every mounted route shall require authentication and enforce a capability guard, with exactly two documented exception lists (unauthenticated-by-necessity; authenticated-but-capability-free)	Implemented — enforced structurally by a route-census test that walks the live Express router tree
TNX-SEC-003	An unrecognized role value in a JWT shall resolve to zero capabilities, never to a default role	Implemented
TNX-SEC-004	A provider API key shall be read from an environment variable only and shall never appear in a log line, an audit row, an HTTP response, or the frontend bundle	Implemented — proven per-provider by a dedicated redaction test and a CI bundle scan
TNX-SEC-005	The analyst who requests a case closure shall never be permitted to approve that same request, except via an explicit ADMIN override capability	Implemented
TNX-SEC-006	The analyst who drafts a notification shall never be permitted to approve it	Implemented
TNX-SEC-007	An AI suggestion's acceptance authority shall never exceed the authority required to perform the same action manually	Implemented
TNX-SEC-008	Login shall return an identical response and timing profile for an unknown email and a known email with a wrong password	Implemented (constant response body; dummy bcrypt comparison runs on both paths)
TNX-SEC-009	No committed file shall be a real .env file, and no credential-shaped literal shall be committed	Implemented — enforced by CI's hygiene job on every push
TNX-SEC-010	The system shall provide a secret manager for provider and signing credentials in production	Production requirement (Part 9) — not implemented; current storage is process environment variables only
TNX-SEC-011	The system shall provide token revocation or a session-invalidation mechanism	Production requirement (Part 9) — not implemented; a role change or account disablement takes effect only when the existing JWT expires
TNX-SEC-012	The system shall have undergone an independent penetration test or third-party security assessment	Not performed for this delivery — stated explicitly in Part 11, not implied by any other control

3.5 Audit requirements

ID	Requirement	Status
TNX-AUD-001	Every write path shall append its own AuditLog event in the same change that performs the write	Implemented — enforced at the service layer, not the controller layer

ID	Requirement	Status
TNX-AUD-002	An audit row shall record actor, role, action, outcome, entity type/id, and a small allow-listed summary — never a raw request body, a provider key, or free-text/PII fields	Implemented
TNX-AUD-003	A failed audit write shall never convert a successful operation into a failure response	Implemented
TNX-AUD-004	A denied authorization attempt shall itself always be audited (<code>authorization.denied</code>) even though the 403 response never names the missing capability to the caller	Implemented
TNX-AUD-005	The system shall provide an in-app audit-log viewer	Production requirement (Part 9) — not implemented; the audit trail is queried directly against the <code>AuditLog</code> table

3.6 Role and capability requirements

ID	Requirement	Status
TNX-ROLE-001	Authorization shall be capability-based and non-hierarchical: no role shall inherit another role's authority implicitly	Implemented
TNX-ROLE-002	Four roles shall exist — ADMIN, ANALYST, REVIEWER, VIEWER — each with an explicit, disjoint capability grant set (ADMIN holds every capability by explicit grant, not by rank)	Implemented
TNX-ROLE-003	VIEWER shall be excluded from all notification read/write capabilities	Implemented
TNX-ROLE-004	The system shall provide an in-app role-assignment or promotion path	Production requirement (Part 9) — not implemented; role assignment requires a direct database change or the seed script

3.7 Requirement families covering ingestion, provider evidence, and AI (cross-references)

To avoid duplicating the detailed behavior already specified in Parts 5–7, the following requirement families are defined here and elaborated there:

- **Report ingestion, validation, deduplication, persistence and recurrence** — TNX-FR-001 through TNX-FR-005 (§3.2); full lifecycle semantics in Part 5.
- **Provider enrichment** — TNX-FR-007 through TNX-FR-009 (§3.2); full per-provider contract in Part 6.
- **Deterministic risk scoring** — TNX-FR-010 (§3.2); full factor model in Part 5.
- **Case management and independent review** — TNX-FR-011 through TNX-FR-015 (§3.2); full workflow in Part 5 and Part 12.
- **Framework mapping** — TNX-FR-016, TNX-FR-017 (§3.2); full rule set in Part 5.

- **Notification drafting, approval and export** — TNX-FR-021 through TNX-FR-026 (§3.2); full lifecycle in Part 5 and Part 12.
- **AI assistance constraints** — TNX-FR-018 through TNX-FR-020 (§3.2), and the dedicated TNX-AI-* family in Part 7.
- **Availability and evidence semantics** ("unknown is never zero", provider evidence vs. proof) — TNX-FR-027 through TNX-FR-029 (§3.2); full model in Part 5.

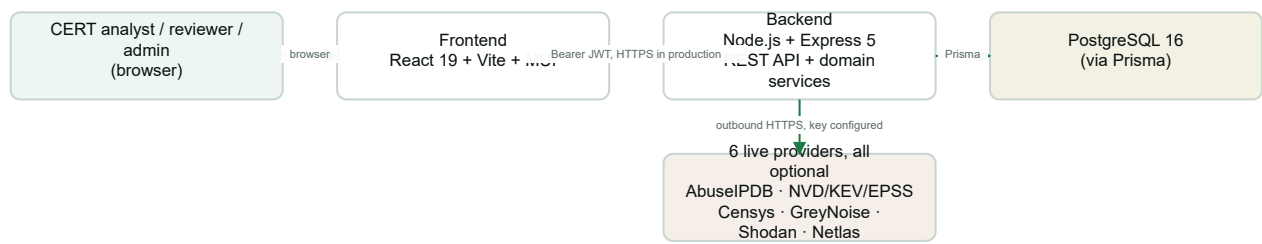
3.8 Requirements traceability

A full traceability matrix (requirement → implementing module → verifying test/evaluator) is provided in Appendix 13.6, generated from the same source-of-truth reconciliation as this section — no requirement above is asserted without a corresponding code path or evaluator named there.

Part 4 — Technical architecture

4.1 System context

ThreatNeXus is a three-tier web application: a React single-page frontend, a Node.js/Express REST API, and a PostgreSQL database accessed through Prisma. Six optional third-party intelligence providers are reachable outbound from the backend only; the frontend never talks to a provider or to PostgreSQL directly, and the backend is the only authorization boundary.



System context

4.2 Containers and components

Component	Path	Notes
REST API	backend/src/routes, backend/src/controllers	Everything mounted under /api — see Appendix 13.2 for the full route catalogue
Domain services	backend/src/services	Ingestion, dedup, ownership, enrichment, risk, workflow, notification, mapping, AI assistance
Provider adapters	backend/src/services/{enrichment,vulnerability,exposure,reputation}	AbuseIPDB, NVD/KEV/EPSS, Censys, GreyNoise, Shodan, Netlas
Persistence	backend/prisma	Prisma + PostgreSQL, 23 additive migrations

Component	Path	Notes
Evaluation harness	eval/	Drives the real services against a disposable database and hand-authored ground truth
Analyst frontend	frontend/src	React 19, Vite (rolldown/oxc), MUI v9, React Router 7, GSAP for restrained motion
Browser exit gate	frontend/e2e	Playwright, Chromium only, 9 spec files against the real stack
CI	.github/workflows/ci.yml	See Part 11

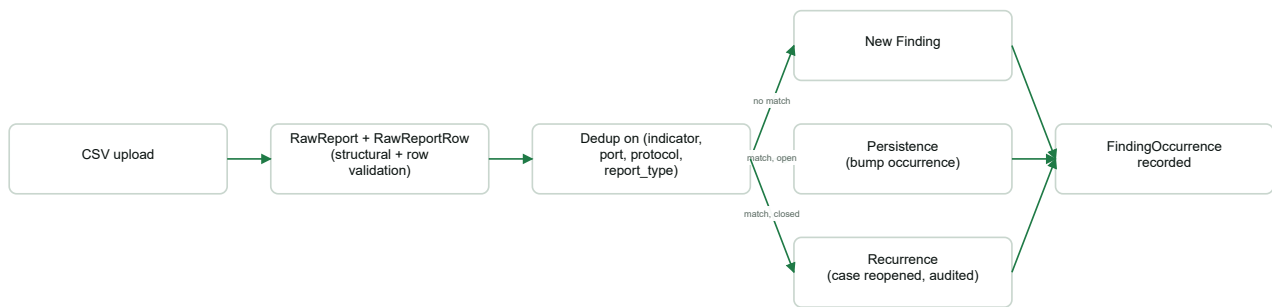
4.3 Backend module map

```

backend/src/
  app.js           Express app assembly – every router mounted here
  server.js        HTTP listener entry point
  config/          env.js (startup validation), prisma.js, rateLimiters.js
  middleware/      authMiddleware, requireRole (capability guard), requestContext,
                   errorHandler, normalizeMulterError
  lib/             roles.js (capability model), validation.js
  routes/ + controllers/ one pair per resource group
  services/
    ingestion/     CSV parsing, row validation, dedup/persistence/recurrence
    ownership/     asset/organization ownership resolution
    enrichment/    IOC reputation (AbuseIPDB/mock), queue, provider registry
    vulnerability/ NVD/KEV/EPSS association and enrichment
    exposure/      Censys, Shodan, Netlas
    reputation/    GreyNoise
    risk/          deterministic Risk v1 scoring engine
    workflow/      case lifecycle, closure, recurrence reopening
    notification/  drafting, revisions, review, export, delivery tracking
    mapping/       ATT&CK / NIST CSF / CIS framework mapping
    ai/            case-level AI mapping-suggestion assistance (Phase 5)
    aiAssist/      Finding-level AI summary/explanation drafts (Phase 8C)
    dashboard/     operationalOverviewService.js – the one read-only snapshot
    auditService.js safeLogAuditEvent – every write path's audit hook

```

4.4 Main data flow

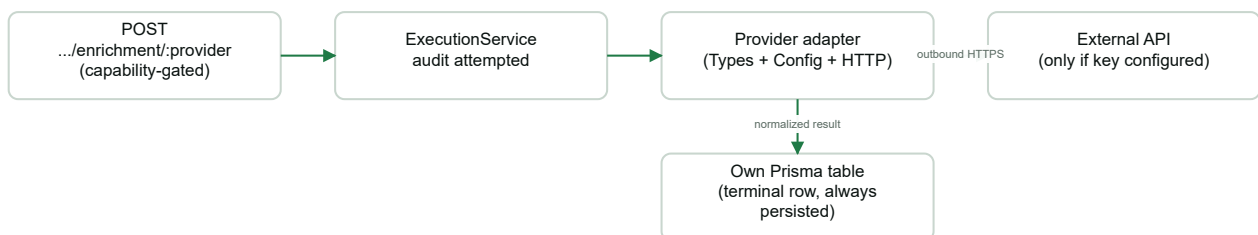


Report-to-finding lifecycle

4.5 Provider adapter pattern

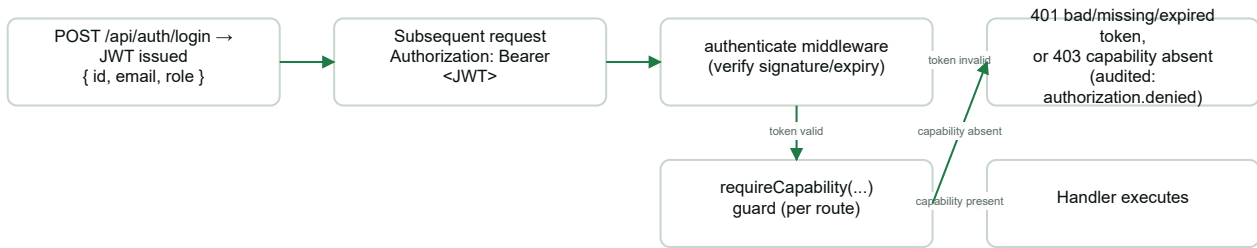
Every live provider (Censys, GreyNoise, Shodan, Netlas — AbuseIPDB and NVD predate the pattern but follow the same shape) is built from four files:

1. `<provider>Types.js` — closed status/error-code taxonomy and one validated result constructor. Every field is bounds-checked; nothing upstream is trusted verbatim.
2. `<provider>Config.js` — pure bounds/defaults for base URL and timeout, shared between startup validation and the provider factory.
3. `<provider>Provider.js` — the HTTP adapter. Composed timeout + caller-abort signal; every HTTP/transport outcome maps to a normalized result; never throws for an *expected* outcome (disabled, rate-limited, not found, malformed, unreachable, timeout are all first-class results).
4. `<provider>ExecutionService.js` — audited orchestration: validate → audit attempted → call the provider → persist the terminal row → audit the outcome. No queue — every provider-execution call in this codebase is synchronous and human-triggered, except AbuseIPDB, which is the one provider wired into automatic post-ingestion enrichment and therefore the one provider modeled as a queue.



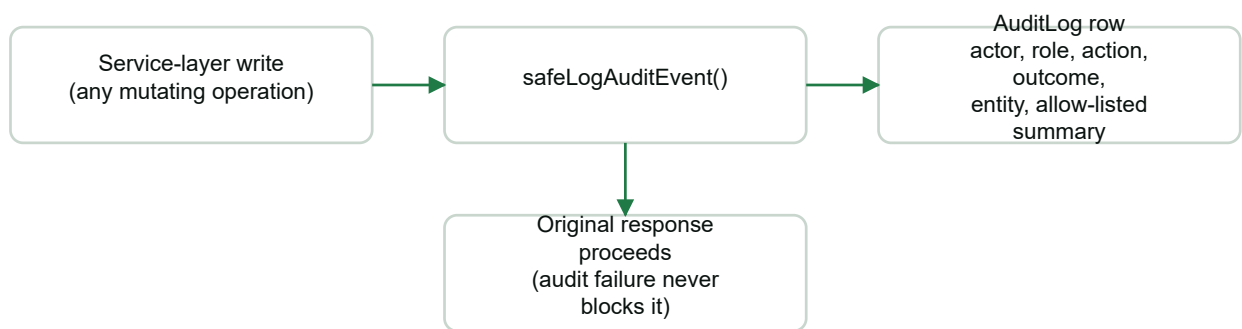
Provider adapter pattern (one provider, generalized)

4.6 Authentication and authorization



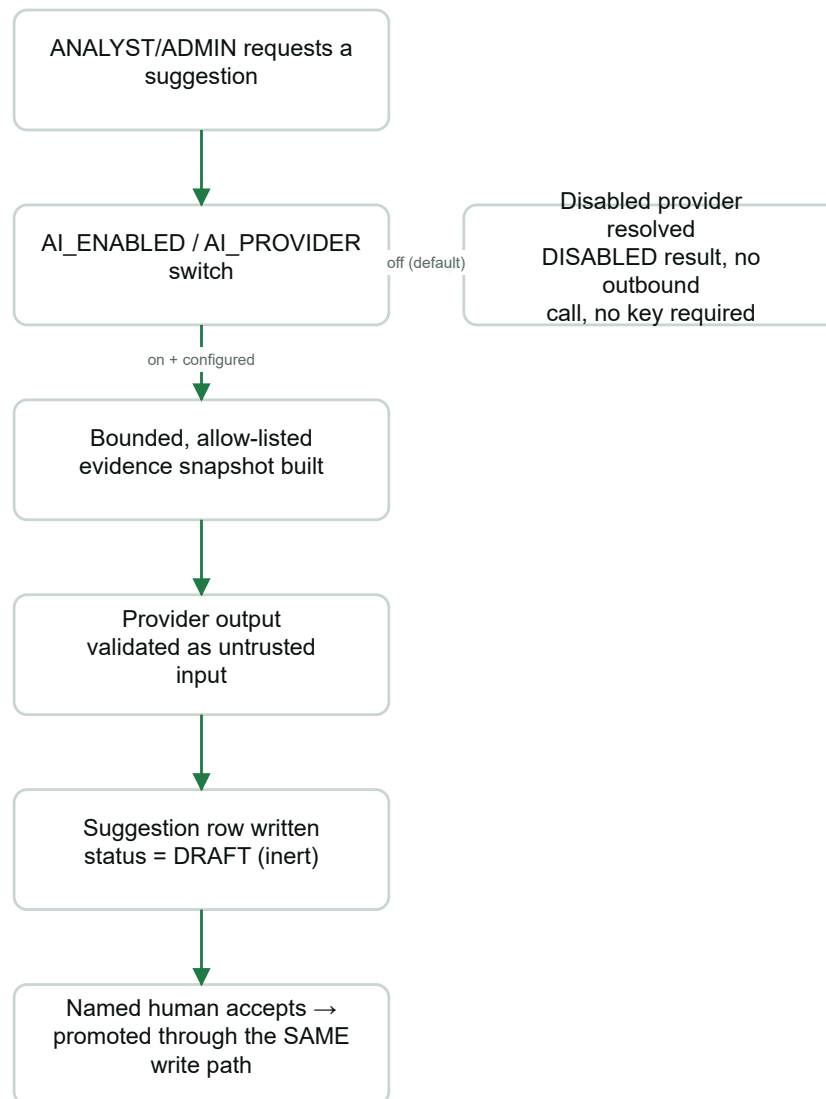
Authentication and authorization

4.7 Audit flow



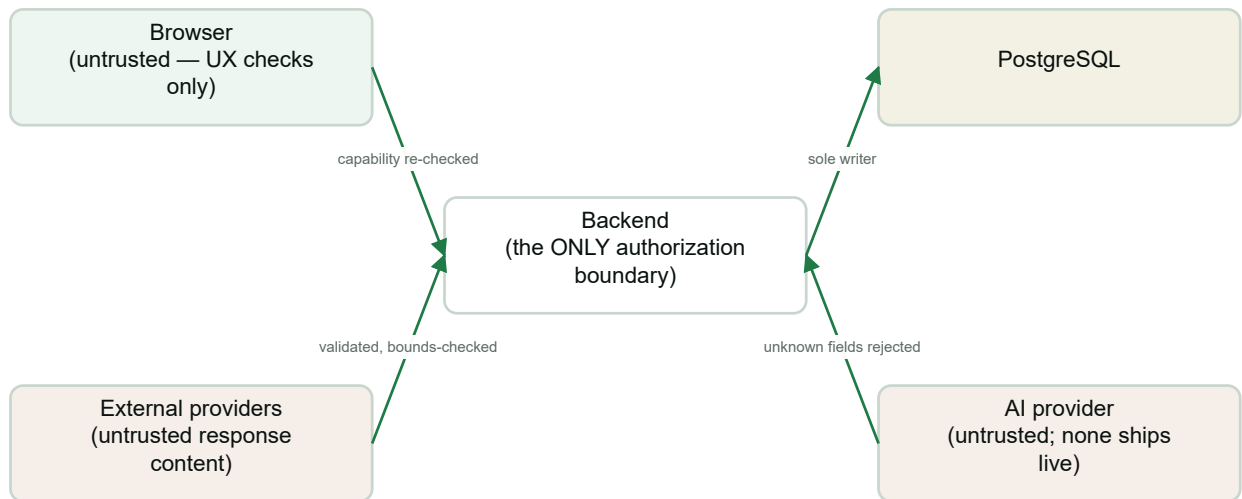
Audit flow

4.8 AI assistance flow



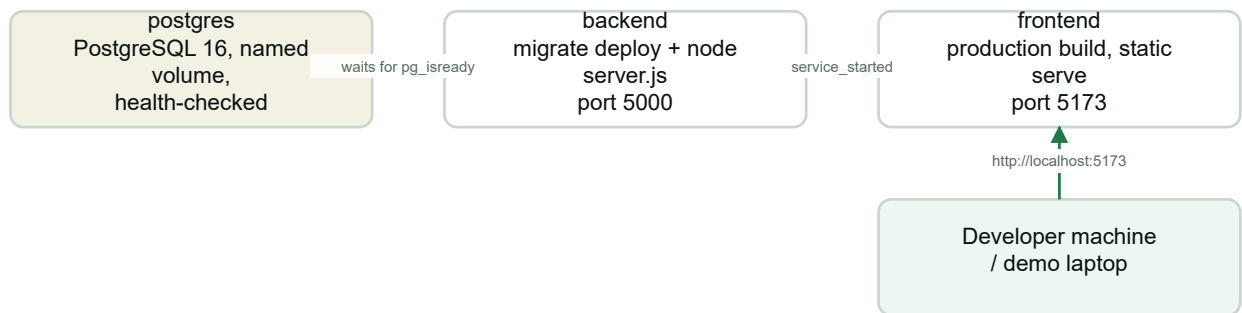
AI assistance flow (either surface)

4.9 Trust boundaries



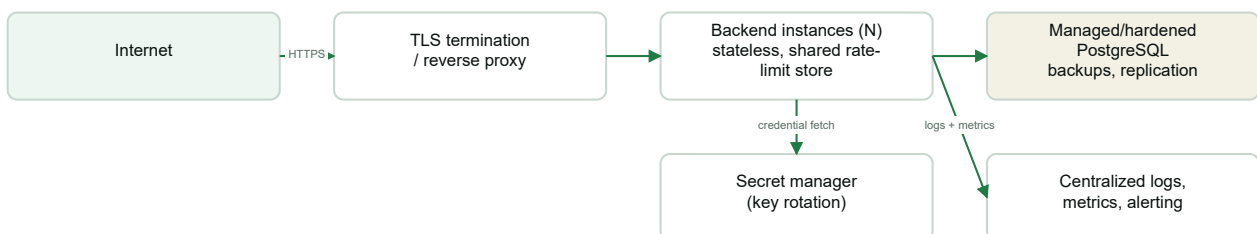
Trust boundaries

4.10 Current Docker topology



Current Docker Compose topology (local / demonstration only)

4.11 Proposed production topology (see Part 9 — not implemented)



Proposed production topology — NOT currently implemented or PKCERT-approved

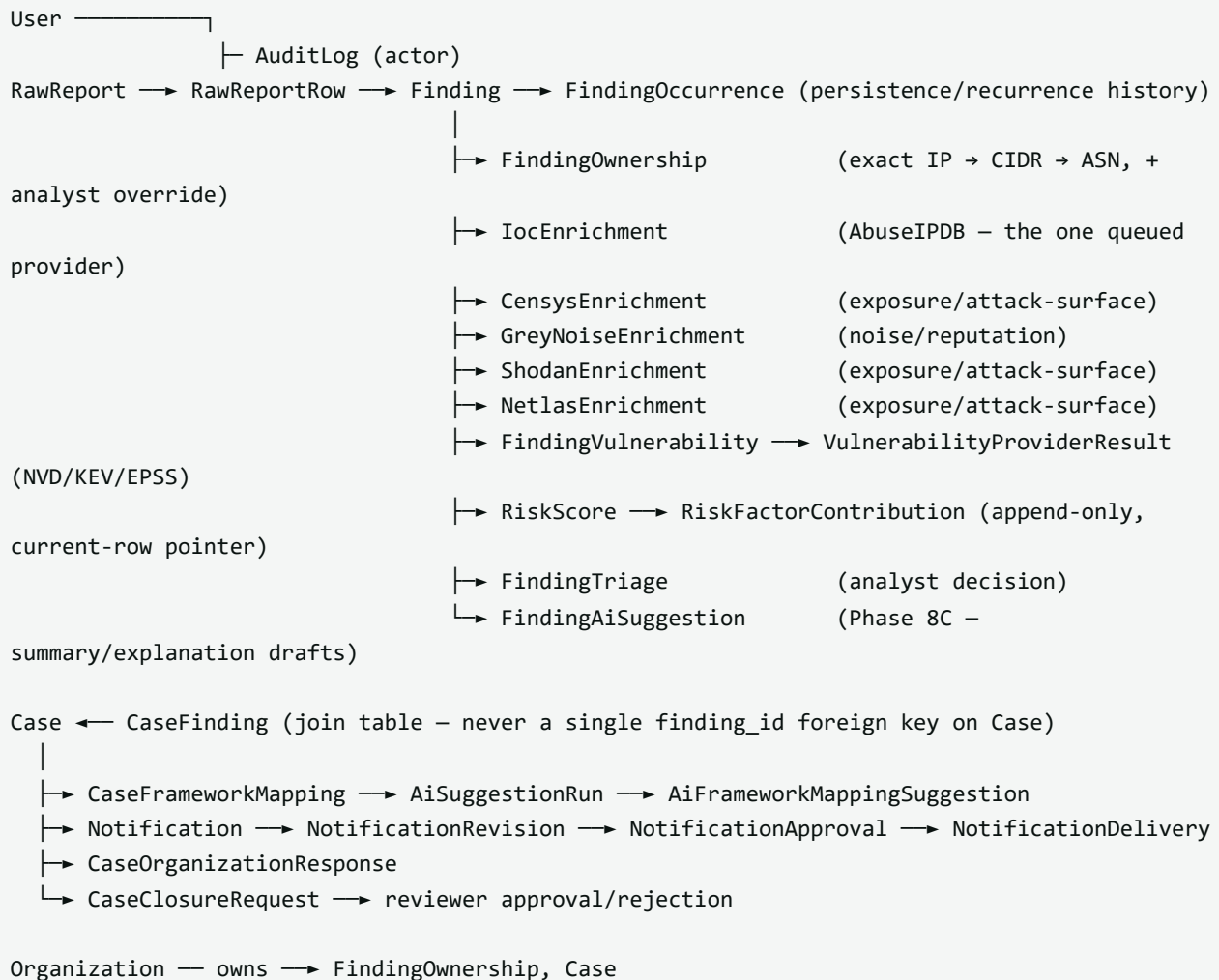
4.12 Technology choices and why

Technology	Why chosen
React 19 / Vite / MUI	Fast dev-server iteration; MUI's accessibility primitives fit a data-dense analyst UI; no charting/map library was added because every figure on the dashboard is a real number or table, never a fabricated visualization
Node.js / Express 5	Matches the existing pre-Phase-0 codebase this project preserves and refactors rather than rewrites (see ../ThreatNeXus-Planning/planning/DECISIONS.md, D-001, which superseded an earlier FastAPI/SQLAlchemy sketch)
Prisma / PostgreSQL 16	Type-safe migrations with a clear, reviewable migration history; PostgreSQL's SERIALIZABLE isolation is used directly for the ownership/case-creation concurrency proofs referenced in Part 11
Docker Compose	Sufficient for local development and demonstration; deliberately not extended to a production orchestration file until a production target is actually approved (Part 9)
Vitest + Supertest	Fast, ESM-native test runner already idiomatic for a Vite-based toolchain; Supertest drives real HTTP requests against the real Express app, not a mocked router
Playwright (Chromium only)	A real-browser exit gate against the real stack; Chromium-only is a stated, deliberate scope limit (§11.7), not an oversight
GitHub Actions	Matches the existing repository host; no separate CI system was introduced
Provider adapter separation (own table per provider)	Each provider's response shape is materially different (a reputation score is not a certificate SAN list); a shared "enrichment" table would either lose fields or accumulate nullable columns nobody's schema needs
Deterministic scoring (no ML, no LLM)	A CERT audience needs to defend a risk number to a constituent; a stored, replayable factor contribution is defensible in a way a model's hidden weights are not
Human approval gates everywhere authority could transfer	The single design decision this whole system is built around: nothing leaves the system, and nothing closes, without a named human's recorded decision

Part 5 — Data model and semantics

5.1 Main entities and relationships

This section names the load-bearing relationships; `backend/prisma/schema.prisma` is authoritative for every column. Twenty-three additive-only migrations built this schema; none has ever altered or dropped an existing column (Part 11 describes how CI enforces this).



5.2 Finding lifecycle

A `Finding` is the dedup unit: one row per (`indicator_value`, `port`, `protocol`, `report_type`). Re-ingesting the same tuple never creates a second row:

- **Persistence** — the `Finding` is already `OPEN` (or has no linked case): `occurrenceCount` increments, `lastSeen` updates, `daysUnresolved` recomputes. No new `Finding` row, no new `Case`.
- **Recurrence** — the `Finding`'s linked `Case` is `CLOSED`: the `Case` transitions `Closed` → `Reopened`, `recurrenceCount` increments, and the reopening is audited. This chain is implemented and proven by `eval:phase1/eval:phase3`, but has no reachable UI/API write path to the `CLOSED` state that triggers it — see §2.9 and Appendix 13.8.

5.3 Risk snapshots and factor contributions

RiskScore + RiskFactorContribution are append-only: currentForFindingId is a nullable @unique pointer to the one current score per Finding (PostgreSQL treats multiple NULLs as distinct, so no raw SQL or partial index is needed to keep history intact while pointing at exactly one current row). Every factor's applicability is one of three values, never collapsed to two:

Value	Meaning
APPLIED	Real evidence was scored, including a legitimate zero
NOT_AVAILABLE	The evidence could not be obtained — a provider failure is never silently rendered as "clean"
NOT_APPLICABLE	The factor cannot apply to this kind of finding

Risk factors include source severity, exposure/service criticality, CVE presence, KEV status, EPSS score, IOC reputation context, persistence, recurrence, days unresolved, and sector criticality. **Ownership confidence is deliberately not a risk factor** — it affects routing and actionability, kept in a separate column with a separate explanation, never blended into the risk number itself.

5.4 Cases, reviews, and separation of duties

A Case binds to exactly one Organization and links Findings through CaseFinding. Closure requires a reviewer other than the requester (enforced server-side, not just in the UI); a REMEDIATED closure additionally requires a prior REMEDIATED CaseOrganizationResponse, refused with REMEDIATED_RESPONSE_REQUIRED otherwise.

5.5 Notifications and immutable revisions

Each edit to a notification produces a new NotificationRevision row; the previous one is never mutated. NotificationApproval binds to the **exact revision id** it reviewed — a later edit does not carry the approval forward, by stored foreign key, not by a comparison a service has to remember to run. Export (GET /api/notifications/:id/export) is refused unless status = Approved and approved_by is non-null. NotificationDelivery is a separate table recording a human's later observation — never inferred from the export event.

5.6 Audit logs

AuditLog is append-only, written exclusively from the service layer (never from a controller), so a service that omits the call fails a test rather than silently skipping an entry. Rows carry actor, role, action, outcome (SUCCESS/FAILURE/DENIED), entity type/id, and a small allow-listed summary — never a raw request body, a provider key, or free-text/PII fields (case descriptions, notification bodies, organization contact details are deliberately excluded).

5.7 Framework mappings

CaseFrameworkMapping rows are append-only (a removal is a new row marking the prior one inactive, never a delete). Every mapping stores framework version, a verbatim evidence quote, an evidence reference, and two **separate** confidence values: evidence_confidence and mapping_confidence. An ATT&CK mapping additionally

requires `evidenceBasis = OBSERVED_BEHAVIOR` and passes a lexical guard checking the rationale describes behavior rather than restating exposure — enforced server-side for both manual and AI-suggested mappings identically. "No applicable mapping" (no-applicable assertion) is a valid, recordable, auditable outcome, not an inferred empty state. **There is no coverage-percentage metric anywhere in this schema or its API** — computing one would require knowing how many references *should* apply, which is unknowable, and printing one would create pressure to force weak mappings.

5.8 Provider-specific enrichment rows

Each provider has its own table, deliberately: `IocEnrichment` (AbuseIPDB, the only queued provider — PENDING/lease/retry/dead-letter, because it is the one provider wired into automatic post-ingestion enrichment), `VulnerabilityProviderResult` (NVD/KEV/EPSS, one row per (provider, identifier)), `CensysEnrichment`, `GreyNoiseEnrichment`, `ShodanEnrichment`, `NetlasEnrichment` — each shaped to what that specific provider actually returns. A shared "enrichment" table would force either losing fields or inventing nullable columns nobody's provider needs; this decision was made and re-confirmed independently at each of Phases 8B, 8D, 8E and 8F rather than assumed the first time (see `docs/ai/SECURITY.md` for the reasoning recorded at each addition).

5.9 AI suggestion records

`AiSuggestionRun` / `AiFrameworkMappingSuggestion` (case-level, Phase 5) and `FindingAiSuggestion` (Finding-level, Phase 8C) are two independent tables, reflecting two independent AI-assistance surfaces — see Part 7. Both store the suggestion, its status (DRAFT/ACCEPTED/REJECTED/EXPIRED), and enough provenance to reconstruct who requested and who decided, without storing the underlying prompt or evidence snapshot in a place that could itself leak sensitive draft text.

5.10 Status vocabularies: UNAVAILABLE, RESTRICTED, stale and absent

Four distinct "this is not a number" states recur across the system, and they are never collapsed into each other or into zero:

State	Where it appears	Meaning
UNAVAILABLE	Dashboard figures, provider status	The value could not be computed or the provider could not answer
RESTRICTED	Dashboard sections	The caller's role may not read this data (a policy fact, not a data fact)
Stale	Provider freshness (STALE vs FRESH)	A stored result exists but is old enough that an analyst should treat it cautiously
Absent	NO_SUCCESSFUL_LOOKUP_RECORDED	No result has ever been stored — different from a stale one and different from a failed one

5.11 Export versus delivery

Notification export produces a file (.eml); `NotificationDelivery` records a separate, later, human-observed fact about what happened after a human sent it by hand. The two counts are never summed on any dashboard

or report — summing them would imply every export was sent, which this system has no way to know and does not claim.

Part 6 — External provider guide

6.1 Shared behavior across all six providers

Every provider result is **supporting context, never proof** — a high AbuseIPDB confidence score, a GreyNoise malicious classification, or an open Shodan port never closes a case, changes a role's authority, or triggers a notification by itself.

- **Failure never blocks ingestion or the workflow it is attached to.** Every adapter maps every expected failure (disabled, invalid key, rate-limited, timeout, unreachable, malformed response, not-found) to a normalized, persisted terminal result — never an unhandled exception, never a silent retry loop, never a fabricated success.
- **A missing key only disables that one provider** (`SKIPPED_DISABLED`). The application starts and every core workflow completes with zero keys configured — proven by `eval:phase7` (replaces `fetch` with a throwing counter and asserts zero calls) and by the offline rehearsal overlay (`docker-compose.offline.yml`, which blackholes every provider host at the DNS level).
- **One shared rate-limit budget.** All six providers' execution routes draw on the same `providerRateLimiter` bucket (`RATE_LIMIT_PROVIDER_MAX`, default 60 per 15-minute window) — a caller cannot obtain a larger effective quota by switching providers. Reading stored results (`GET`) is never rate-limited; only causing new provider spend is.
- **No secret ever reaches a log line, an audit row, or the frontend.** Proven per provider by a dedicated redaction test, and by CI's own scan of the production frontend bundle.
- **Own Prisma table per provider**, not a shared "enrichment" table — see §5.8.
- **No queue for the five providers added after AbuseIPDB.** Censys, GreyNoise, Shodan, and Netlas are synchronous, human-triggered, single-attempt lookups: a human clicks "look up," one HTTP call happens, one row is persisted.

6.2 AbuseIPDB — IP reputation (the required first provider)

Purpose	Confidence score, report count, and whitelist status for an IPv4 indicator
Domain	IOC reputation
Env vars	<code>IOC_ENRICHMENT_PROVIDER</code> (<code>mock/abuseipdb</code>), <code>ABUSEIPDB_API_KEY</code> , <code>ABUSEIPDB_BASE_URL</code> , <code>ABUSEIPDB_TIMEOUT_MS</code> , <code>ABUSEIPDB_MAX_AGE_DAYS</code> , <code>ABUSEIPDB_CACHE_TTL_HOURS</code>
Auth	API key header
Storage	<code>IocEnrichment</code> — the only provider table modeled as a queue
Automatic vs. human-triggered	Automatic, on ingestion — the only provider wired this way
Offline behavior	<code>mock</code> (the default) needs no key; every automated test and evaluator uses it
Live smoke	None shipped specifically for this provider — exercised only through the mock in tests/evaluators

6.3 NVD, CISA KEV, FIRST EPSS — vulnerability metadata

Purpose	CVE metadata (NVD), known-exploited status (KEV), exploit-prediction score (EPSS)
Domain	Vulnerability enrichment — a separate path from IOC reputation; neither substitutes for the other
Env vars	NVD_API_KEY (optional), NVD_BASE_URL, NVD_TIMEOUT_MS, CISA_KEV_URL, CISA_KEV_TIMEOUT_MS, FIRST_EPSS_BASE_URL, FIRST_EPSS_TIMEOUT_MS
Auth	NVD: optional API key (keyless works at a lower public rate limit); KEV and EPSS: none, public catalogues
Storage	VulnerabilityProviderResult, one row per (provider, identifier)
Live smoke	LIVE_NVD_SMOKE=1 npm run smoke:nvd --prefix backend — one lookup against a permanently published CVE. Not run for this delivery.
Attribution	<i>"This product uses the NVD API but is not endorsed or certified by the NVD."</i>

A missing NVD_API_KEY reports as KEYLESS_PUBLIC_RATE_LIMIT, not NOT_CONFIGURED — a genuinely different, still-valid mode from a key-required provider, and the dashboard says so explicitly.

6.4 Censys — internet exposure / attack surface

Purpose	Open services and autonomous-system ownership for an IPv4 indicator
Domain	Exposure / attack surface
Env vars	CENSYS_PAT, CENSYS_ORG_ID (optional, multi-org accounts), CENSYS_BASE_URL, CENSYS_TIMEOUT_MS
Auth	Authorization: Bearer <PAT> against Censys's current Platform API (api.platform.censys.io) — not the legacy Search v2 API
Storage	CensysEnrichment
Route	GET/POST /api/findings/:id/enrichment/censys
Live smoke	LIVE_CENSYS_SMOKE=1 npm run smoke:censys --prefix backend against 1.1.1.1. Not run this delivery.

6.5 GreyNoise — internet noise / scanning context

Purpose	Whether an IPv4 address is known internet background noise, plus a benign/malicious/unknown classification
Domain	Reputation — its own dashboard array, distinct from exposure
Env vars	GREYNOISE_API_KEY, GREYNOISE_BASE_URL, GREYNOISE_TIMEOUT_MS
Auth	key header (Community API tier)
Storage	GreyNoiseEnrichment
Route	GET/POST /api/findings/:id/enrichment/greynoise
Live smoke	LIVE_GREYNOISE_SMOKE=1 npm run smoke:greynoise --prefix backend against 1.1.1.1. Not run this delivery.

GreyNoise's classification is a closed vocabulary; a value outside benign/malicious/unknown is normalized to null, never passed through as an invented fourth value.

6.6 Shodan — exposed service / banner / port intelligence

Purpose	Hostnames, organization/ISP, geo, per-service product+version banners, CVE identifiers for an IPv4 indicator
Domain	Exposure / attack surface — joins the same dashboard array as Censys
Env vars	SHODAN_API_KEY, SHODAN_BASE_URL, SHODAN_TIMEOUT_MS
Auth	key query parameter — Shodan's own documented scheme, no header option
Storage	ShodanEnrichment
Route	GET/POST /api/findings/:id/enrichment/shodan
Live smoke	LIVE_SHODAN_SMOKE=1 npm run smoke:shodan --prefix backend against 8.8.8.8. Not run this delivery.

A vulns entry that doesn't match Shodan's own CVE-YYYY-NNNN+ format is dropped, never passed through as an invented CVE.

6.7 Netlas — cross-source attack-surface / DNS / certificate intelligence

Purpose	Reverse DNS, associated domains, ASN/organization ownership, open ports, software banners, TLS certificate subject/issuer/SAN for an IPv4 indicator
Domain	Exposure / attack surface — third entry in the same dashboard array as Censys and Shodan
Env vars	NETLAS_API_KEY, NETLAS_BASE_URL, NETLAS_TIMEOUT_MS
Auth	Authorization: Bearer <key> (RFC 6750) — the older X-API-Key header is deprecated and unused here
Storage	NetlasEnrichment
Route	GET/POST /api/findings/:id/enrichment/netlas
Live smoke	LIVE_NETLAS_SMOKE=1 npm run smoke:netlas --prefix backend against 8.8.8.8. Not run this delivery.

Netlas's ports[] and products[] are stored as two separate arrays because the provider's response carries no positional/key correlation between them — merging would fabricate a join the evidence does not support. Netlas's own 402 ("out of subscription plan limits") is treated as rate-limited, alongside 429.

6.8 Providers not integrated

- **Shadowserver.** ThreatNeXus *consumes* Shadowserver-style report files as its input format, but there is no live scheduled Shadowserver API ingestion — reports are uploaded as files. A live feed is out of scope for this delivery, pending API access/licensing this project does not currently hold.

- **VirusTotal, OTX, MISP.** Not integrated — no adapter, no env var, no code path references them.
Recommended as the natural seventh provider if one is wanted, following the same adapter pattern.

6.9 Quota protection

The shared `providerRateLimiter` budget is the only quota-protection mechanism in this codebase — there is no per-provider daily cap, no cost-based throttling, and no admin-configurable spend ceiling beyond the one shared rate-limit bucket. A production deployment handling meaningfully higher volume would want per-provider budget tracking; this is listed in Part 9's gap matrix, not silently assumed to exist.

Part 7 — AI governance

7.1 The one operator switch

```
AI_ENABLED=false      # shipped default
AI_PROVIDER=null      # shipped default; no real provider name resolves to anything but
"disabled"
```

Both AI surfaces read the same switch — there is no per-feature toggle. With `AI_ENABLED=false`, the disabled provider is resolved regardless of `AI_PROVIDER`, and no suggestion request ever reaches an outbound call. `eval:phase7` proves this by replacing `fetch` with a throwing counter and asserting it is never invoked with AI off.

7.2 No live AI provider ships

`AI_PROVIDER` accepts a name, but the only two names either registry resolves to a real factory are disabled and a test-only mock — and mock is reachable *only* with an explicit `allowMockProvider: true` flag that no production code path ever passes. Setting `AI_PROVIDER` to anything else (a real model name, a typo) resolves to `null`, surfacing as `AI_PROVIDER_NOT_AVAILABLE` — never a silent fallback to mock output presented as if a real model answered it. Turning AI "on" today means turning on a provider that safely returns nothing.

7.3 The two AI surfaces

	Case-level mapping suggestions (Phase 5)	Finding-level narrative drafts (Phase 8C)
What it proposes	ATT&CK / NIST CSF / CIS mapping candidates for a case	A SUMMARY or EXPLANATION draft for one Finding
Module	backend/src/services/ai/	backend/src/services/aiAssist/
Storage	AiSuggestionRun, AiFrameworkMappingSuggestion	FindingAiSuggestion
Evidence snapshot	caseEvidenceSnapshot.js — named, explicit columns only	findingEvidenceSnapshot.js — same discipline, structurally excludes indicator value, port, protocol, and organization contact detail
Who requests	ADMIN, ANALYST	ADMIN, ANALYST
Who decides	ADMIN, ANALYST (decide:ai-mapping-suggestions)	ADMIN, REVIEWER (review:ai-suggestions)
What acceptance does	Promotes through the same write path a manual mapping uses	Flips only the suggestion's own review state — nothing downstream to promote into
Frontend surface	AI mapping panel on the case framework workspace	FindingAiAssistPanel.jsx on the Finding-detail page

The asymmetry in "who decides" is deliberate: an analyst may decide a mapping suggestion because an analyst already holds the authority to write the same mapping manually (`manage:framework-mappings`); an analyst may **not** decide a Finding-level draft, because that decision belongs structurally to the same role that approves notifications and closures (REVIEWER), matching the notification-approval separation of duties.

7.4 Why acceptance can never exceed manual authority

This is the one rule the entire AI governance model exists to enforce: the capability that decides an AI suggestion is granted to exactly the same roles as the capability that performs the same action manually. If a role could approve an AI suggestion without also being allowed to perform the equivalent manual action, the AI path would grant authority the manual path denies — checked in code (identical capability grants in `roles.js`), not only asserted in this document.

7.5 What AI is structurally unable to do

The provider contract has exactly one method and it returns data. A provider factory receives no Prisma client, no transaction handle, no repository, no HTTP session, and no capability token — there is nothing on which it could score, approve, close, reopen, export, notify, enrich, or write anything at all. AI in this system **never**:

- makes a final framework-mapping decision, a risk-scoring decision, or a case/notification decision
- sends a notification, exports anything, or triggers delivery
- closes, reopens, or reclassifies a Finding or a Case
- gains write access to anything beyond its own suggestion row
- has its output trusted verbatim

7.6 AI output is untrusted input

A provider response is treated exactly as a request body from an anonymous client would be: unknown keys are **rejected**, not silently dropped; every value is type-and-bound checked; mapping content clears the same rules a hand-written mapping clears, including the server-side ATT&CK evidence rule. A candidate that fails is **discarded and counted** — never repaired, never coerced, never partially persisted, never shown to an analyst. For Finding-level drafts, only `text` and `evidenceReferences` are ever read off a provider result, and `evidenceReferences` must name only fields present in the snapshot's own closed allow-list.

7.7 Staleness

If a Finding's evidence has changed since a draft was generated, an accept attempt transitions the draft to EXPIRED and is refused — never silently re-derived against new evidence. Rejecting is always allowed, unconditionally.

7.8 Prompt-injection handling

Analyst-supplied request context and provider-returned text are plain string values on a data object; nothing in this codebase parses instructions out of either one. A dedicated test drives an adversarial payload (text designed to look like an instruction to the system) through the real path end to end and asserts: no Finding mutation occurs, and nothing is auto-accepted. The defense is structural — the string is never interpreted as anything but display text — rather than a filter a cleverer payload could defeat.

7.9 Human review and separation of duties

A suggestion is inert until a named human reviewer accepts or rejects it. Every AI action is audited: `ai.suggestion.requested`, `.generated`, `.failed`, `.accepted`, `.rejected`, `.unavailable` — actor and role, provider name, and a closed reason code. **Never the proposed text, the evidence snapshot, or any internal fingerprint** — an auditor can reconstruct who requested what and who decided it without the audit log itself becoming a second place sensitive draft text could leak from.

7.10 Frontend behavior

Role visibility renders from capabilities the server actually returned at login, never a locally hardcoded role table — UX convenience only; the backend re-checks every capability regardless. A draft is **never rendered as a finding fact**: every draft carries its own status badge (label, icon, and color together, never color alone), evidence references as human-readable tags from a closed allow-list, and an advisory note that accepting only records a human decision. No raw provider error, prompt, or backend exception text ever reaches the browser.

7.11 Actions AI is structurally unable to perform (consolidated)

Score a finding · approve its own suggestion · create an ownership mapping · attach a CVE · close, reopen, or resolve a case · approve a closure · create, approve, export, or deliver a notification · record an organization response · send email · run enrichment · scan anything · create an active framework mapping without human promotion.

7.12 Limitations, stated honestly

- **No live AI provider ships.** Enabling `AI_ENABLED=true` today activates a provider that resolves to "disabled" regardless — pointing at a real model requires writing a new provider adapter first, a deliberate, out-of-scope decision.
 - **No catalogue validation on AI mapping candidates.** A syntactically valid but non-existent ATT&CK technique id passes shape validation and reaches review, because no pinned catalogue existed when Phase 5 shipped (Phase 6.3 later added one for the manual/AI-shared navigator; the mapping *suggestion* pipeline's validation predates it).
 - **A human must still read every suggestion.** Nothing measures suggestion quality, flags a suspicious accept/reject pattern, or second-guesses a reviewer's decision. The audit trail makes decisions traceable; it does not make them automatically correct.
-

Part 8 — Current installation and deployment manual

8.1 Supported use

Docker Compose, for a developer's own machine or a live demonstration. There is no production compose file and no deployed environment. This section documents what exists; Part 9 proposes, and explicitly labels as unimplemented, what a production rollout would additionally require.

8.2 Required software

Requirement	Notes
Docker Desktop (Windows) or Docker Engine + Compose plugin	Runs all three services
Git	To clone the repository
A way to generate a random 32+ character string for JWT_SECRET	PowerShell example below; openssl rand -base64 48 on POSIX
Optional: API keys for any of the six live providers	None is required to run the full application (Part 6)

Hardware sizing is **not formally benchmarked** — no load test has been run against this codebase. Demonstration-workstation guidance only: a modern laptop capable of running Docker Desktop with 4+ CPU cores and 8+ GB RAM available to Docker has run the full stack, migrations, and demo seed comfortably in this project's own development environment. This is an observation, not a sizing requirement.

8.3 Repository setup (Windows / PowerShell — the primary presentation machine)

```
git clone https://github.com/haiderchattha99-ali/ThreatNeXus.git
Set-Location ThreatNeXus
```

No .env file is required to start the stack — docker-compose.yml reads shell-exported variables with safe defaults, except JWT_SECRET, which has no default and is required.

POSIX equivalent:

```
git clone https://github.com/haiderchattha99-ali/ThreatNeXus.git
cd ThreatNeXus
```

8.4 Environment-variable categories

backend/.env.example is the authoritative, always-current list — placeholders only, never a real key.

Category	Variables	Required?
Core	NODE_ENV, PORT, DATABASE_URL, JWT_SECRET, JWT_EXPIRES_IN, CORS_ORIGIN	JWT_SECRET required, no default; everything else defaulted
Uploads	UPLOAD_MAX_BYTES, REPORT_MAX_ROWS	Defaulted
IOC reputation	IOC_ENRICHMENT_PROVIDER, ABUSEIPDB_API_KEY, ABUSEIPDB_BASE_URL, ABUSEIPDB_TIMEOUT_MS, ABUSEIPDB_MAX_AGE_DAYS, ABUSEIPDB_CACHE_TTL_HOURS	Optional — mock needs nothing
Vulnerability	NVD_API_KEY, NVD_BASE_URL, NVD_TIMEOUT_MS, CISA_KEY_URL, CISA_KEY_TIMEOUT_MS, FIRST_EPSS_BASE_URL, FIRST_EPSS_TIMEOUT_MS	Optional — NVD works keyless at a lower rate limit; KEV/EPSS need no key
Exposure	CENSYS_PAT, CENSYS_ORG_ID, SHODAN_API_KEY, NETLAS_API_KEY (+ each provider's _BASE_URL/_TIMEOUT_MS)	Optional
Reputation	GREYNOISE_API_KEY (+ _BASE_URL/_TIMEOUT_MS)	Optional
AI	AI_ENABLED (default false), AI_PROVIDER (default null)	Optional — Part 7
Rate limiting	RATE_LIMIT_ENABLED, RATE_LIMIT_AUTH_MAX, RATE_LIMIT_AUTH_WINDOW_MS, RATE_LIMIT_UPLOAD_MAX, RATE_LIMIT_UPLOAD_WINDOW_MS, RATE_LIMIT_PROVIDER_MAX, RATE_LIMIT_PROVIDER_WINDOW_MS	Defaulted
Demo/seed only	SEED_USER_PASSWORD, DEMO_MODE, DEMO_USER_PASSWORD	Never set outside a seed run

Never write a real key into `docker-compose.yml`, `docker-compose.offline.yml`, or any tracked file. Every provider key is passed through from the invoking shell.

8.5 JWT secret generation

PowerShell (the primary path):

```
$jwt = [Convert]::ToBase64String((1..48 | ForEach-Object { Get-Random -Maximum 256 })))
```

POSIX:

```
openssl rand -base64 48
```

The value must be at least 32 characters and must not be a recognizable placeholder — startup validation rejects an obvious default.

8.6 Starting the stack

PowerShell:

```
$env:JWT_SECRET = $jwt
docker compose up --build
```

POSIX:

```
JWT_SECRET="$(openssl rand -base64 48)" docker compose up --build
```

Three services come up: **postgres** (PostgreSQL 16, named volume, pg_isready health check the backend waits on), **backend** (applies every pending migration via `prisma migrate deploy`, then starts the API on port 5000), **frontend** (a production build served on port 5173, pointed at `VITE_API_BASE_URL`).

8.7 Optional provider configuration

PowerShell pattern for any provider:

```
$env:CENSYS_PAT = '<your-pat>'
$env:JWT_SECRET = $jwt
docker compose up -d
```

A provider with no key is not an error state — it is `NOT_CONFIGURED/SKIPPED_DISABLED`, a normal, persisted, honestly-labeled outcome.

8.8 Database creation and Prisma migrations

Migrations apply automatically on container start. There are 23 migrations, all additive. To apply manually against a running database:

```
docker compose exec backend npx prisma migrate deploy
```

To check status without modifying anything:

```
docker compose exec backend npx prisma migrate status
```

8.9 Seed users and demo seed

```
docker compose exec -e SEED_USER_PASSWORD='<a-strong-local-password>' backend npm run
seed:users
docker compose exec -e DEMO_MODE=true -e DEMO_USER_PASSWORD='<same password>' backend npm run
seed:demo
```

`seed:users` creates exactly four accounts (`admin@threatnexus.local`, `analyst@threatnexus.local`, `reviewer@threatnexus.local`, `viewer@threatnexus.local`) and touches no other row. `seed:demo` drives the application's own REST API with real per-role JWTs, so the analyst's own self-approval attempt on a case closure is issued and refused with 403 **during the seed run itself** — the demonstration proves separation of

duties rather than asserting it. Neither script prints a password; both refuse `NODE_ENV=production` without an explicit override; `seed:demo` is idempotent.

8.10 Online mode, offline rehearsal mode, live-provider mode

Mode	Command	Behavior
Online (default)	<code>docker compose up --build</code>	Providers reach the real internet only if a key is configured
Offline rehearsal	<code>docker compose -f docker-compose.yml -f docker-compose.offline.yml up -d</code>	Every provider host is blackholed at the DNS level inside the container; workflow behavior is identical — enrichment records "unavailable" instead of a live result
Live-provider mode	Any online run with one or more provider keys exported	That provider's lookups reach the real API; still fails safely if the call errors

8.11 Health checks

- **postgres:** `pg_isready -U threatnexus -d threatnexus`, every 5s, 10 retries.
- **backend:** an HTTP probe against the root path, every 10s, 30s start period.
- **frontend:** depends on backend: `service_started` (not `service_healthy` — a static build server does not need the API fully warmed to start).

8.12 Startup, shutdown, reset

```
docker compose up --build      # start (builds if needed)
docker compose up -d --build   # start in background
docker compose down            # stop, keep data
docker compose down -v         # stop AND delete all data
docker compose restart backend # restart one service without rebuilding
```

8.13 Backup and restore

There is no automated backup mechanism shipped with this project.

```
docker compose exec postgres pg_dump -U threatnexus threatnexus > backup.sql
```

Restore into a fresh volume: `docker compose down -v`, bring the stack back up, then:

```
Get-Content backup.sql | docker compose exec -T postgres psql -U threatnexus threatnexus
```

`docker compose down` (without `-v`) never touches the named volume; only an explicit `-v` does.

8.14 Troubleshooting

Symptom	Cause	Fix
<code>docker compose up</code> exits immediately citing <code>JWT_SECRET</code>	The variable was not exported	Set it and retry (§8.5–8.6)
Bind for <code>0.0.0.0:5432</code> failed: port is already allocated	An older container is still holding the port	Stop the conflicting container or local Postgres
Fresh Postgres reports empty <code>NetworkSettings.Networks</code> <code>{}</code>	A stale Docker network reference from a previous cycle	<code>docker compose down -v</code> ; <code>docker network prune</code> , then bring the stack up again
<code>seed:demo</code> reports findings <code>total=0</code>	The <code>./data:/app/data:ro</code> fixture mount is missing	Confirm the volume mount, rebuild
Backend never becomes healthy	Migrations taking longer than the 30s start period, or Postgres never became healthy	Check <code>docker compose logs backend</code> and <code>docker compose logs postgres</code>
A provider row is always <code>FAILED/SKIPPED_DISABLED</code>	No key configured, or the key was rejected	Check <code>GET /api/dashboard/overview</code> → <code>sections.providers</code> , or Part 6 for that provider's exact env var names

8.15 What must never be committed

Any `.env` file other than `backend/.env.example` (CI fails the build if one is found) · any real provider API key, JWT secret, or database password anywhere · `frontend/dist` or any build output · `node_modules` in either `backend/` or `frontend/`.

Part 9 — Proposed production deployment blueprint

Proposed production architecture — not currently implemented or PKCERT-approved. Everything in this Part describes what a production rollout would require, not what exists today. No statement here should be read as a claim that ThreatNeXus has been deployed to production, hardened for constituent-facing traffic, or approved by PKCERT/NCERT for such use.

9.1 Production network topology (proposed)

A reverse proxy terminating TLS in front of a stateless backend fleet, talking to a managed or hardened PostgreSQL instance, with a secret manager supplying credentials at boot rather than through process environment variables set by hand. See the diagram in §4.11.

9.2 Gap matrix

Area	Current prototype state	Production requirement	Gap	Priority	Evidence/source
TLS / HTTPS	None — local HTTP only	TLS termination at a reverse proxy or load balancer	Full	High	<code>docker-compose.yml</code> exposes plain HTTP ports
Reverse proxy / DNS	None	A reverse proxy with a real domain and certificate	Full	High	No proxy config exists in the repository
Database	Single-container PostgreSQL, named volume, no replication	Managed or hardened PostgreSQL with backups and replication	Full	High	<code>docker-compose.yml</code> §8.6
Persistent storage	Docker named volume on one host	Durable, backed-up storage with a tested restore path	Full	High	§8.13 — backup is a manual <code>pg_dump</code> , restore untested against this deployment
Secret management	Process environment variables, developer-set	A secret manager with rotation	Full	High	<code>docs/ai/SECURITY.md</code> — API keys via env vars only
API key rotation	Manual (edit env, restart)	Automated or scheduled rotation	Full	Medium	Part 6
Centralized logging	Per-container <code>docker compose</code> logs	Aggregated, retained, searchable logs	Full	Medium	§10.2
Metrics	None	Application and infrastructure metrics	Full	Medium	No metrics endpoint exists

Area	Current prototype state	Production requirement	Gap	Priority	Evidence/source
Alerting	None	Paging/notification on failure conditions	Full	Medium	None exists
Shared rate-limit storage	In-process, per-Node-process counters	A shared store (Redis or equivalent) for multi-instance limits	Full	High	TNX-NFR-010
Multiple backend instances	Single instance	A stateless backend fleet behind a load balancer	Full	High	Current Compose brings up exactly one backend container
Health/readiness probes	A basic HTTP root-path check	Distinct liveness/readiness semantics for an orchestrator	Partial	Medium	§8.11
Migration rollout	prisma migrate deploy on container start	A controlled rollout (canary, maintenance window, rollback plan)	Partial	Medium	§8.8
Release and rollback	None — manual git operations	A documented release/rollback procedure	Full	Medium	§10.9
Backup schedule	Manual, on demand	Scheduled, automated, monitored backups	Full	High	§8.13
Restore testing	Never performed	A periodically rehearsed restore drill	Full	High	§8.13
Disaster recovery	None	A documented DR plan	Full	Medium	Not attempted in this delivery
RPO / RTO	Undefined	Explicit targets set by a stakeholder decision	Full	Medium	Deliberately left as a stakeholder decision, not invented here
Data retention / privacy	Undefined beyond "synthetic data only"	A retention policy for real constituent data	Full	High	This project has never held real data (§1.6)
Token revocation / SSO	None — JWT expires naturally after JWT_EXPIRES_IN	Revocation or an SSO/identity-provider integration	Full	Medium	TNX-SEC-011
Vulnerability management	Manual dependency review	A scheduled dependency/vulnerability scanning process	Partial	Medium	CI's secret scan exists; no dependency-CVE scan is wired in
Security monitoring	None beyond audit logs	Active security monitoring / SIEM integration	Full	Medium	Explicitly out of scope by design (§2.7)
Incident response	None formalized	A documented incident-response procedure	Full	Medium	Not attempted in this delivery

Area	Current prototype state	Production requirement	Gap	Priority	Evidence/source
Independent security assessment	Not performed	A third-party penetration test	Full	High	TNX-SEC-012
Production acceptance criteria	None defined	A signed-off checklist before any production cutover	Full	High	Appendix 13.7 provides a starting checklist, not a completed one

9.3 Production acceptance criteria (proposed, not yet met)

A move toward production should not proceed until, at minimum: an independent security assessment has been completed and its findings closed or explicitly accepted; TLS, a secret manager, and a shared rate-limit store are in place; a tested backup/restore drill has succeeded at least once; RPO/RTO targets have been set by a stakeholder with the authority to accept the associated risk; and a rollback procedure has been rehearsed, not merely written down. None of these has occurred as of this document's date.

Part 10 — Operations and maintenance

10.1 Startup / shutdown SOP

```
$env:JWT_SECRET = $jwt
docker compose up --build          # attended, streams logs from all three services
docker compose up -d --build       # unattended
docker compose down                # stop, keep data
docker compose down -v             # stop, delete all data – irreversible
docker compose restart backend     # restart one service without rebuilding
```

10.2 Service health and logs

```
docker compose logs -f backend
docker compose logs -f postgres
docker compose logs --tail=200 backend
```

The backend logs its own `requestId` on every request (`requestContext` middleware), echoed back as the `x-Request-Id` response header — the correlation key between a frontend-reported error and a specific backend log line.

10.3 Database and migration checks

```
docker compose exec backend npx prisma migrate status    # never modifies anything
docker compose exec postgres psql -U threatnexus -d threatnexus
docker compose exec backend npx prisma migrate deploy    # applies any pending migration, non-destructive
```

10.4 Provider status checks

```
curl.exe -s -H "Authorization: Bearer $TOKEN" http://localhost:5000/api/dashboard/overview |
jq '.data.sections.providers'
```

This call makes **zero live provider requests** — it derives status from configuration plus stored rows. To verify one live provider is actually reachable (never automated, never run in CI):

```
$env:LIVE_CENSYS_SMOKE = '1'; npm run smoke:censys --prefix backend
```

10.5 Rate-limit events

Three independent buckets — see §3.3 (TNX-NFR-006) and Part 6, §6.9. A rate-limited caller receives 429; reading data is never rate-limited. Known legitimate override: the Chromium browser suite signs in dozens of

times across four roles in a few minutes, which the default auth budget (30/15 min) correctly refuses as credential-stuffing-shaped traffic — CI raises `RATE_LIMIT_AUTH_MAX` to 1000 for its own disposable backend only, never for a real deployment's default.

10.6 Key expiry and provider failures

A provider stuck at `FAILED/INVALID_KEY` almost always means a wrong, expired, or wrong-scheme key — check Part 6 for that provider's exact auth mechanism, then re-check the key value (never logged, so it cannot be recovered from a log line).

10.7 Backup and restoration

See §8.13. There is no automated schedule; a production deployment would need one (Part 9, §9.2).

10.8 Offline rehearsal

Run before any live presentation with uncertain network connectivity — the point of a rehearsal is to be surprised in private, not in front of an audience.

```
$env:JWT_SECRET = $jwt
docker compose -f docker-compose.yml -f docker-compose.offline.yml up -d
```

10.9 Common failures and recovery

Symptom	Likely cause	Recovery
<code>docker compose up</code> refuses to start	<code>JWT_SECRET</code> not exported	Set it and retry
Port 5432 already in use	An old Postgres container is still bound	Stop the offending container
Fresh Postgres gets empty <code>NetworkSettings.Networks {}</code>	Stale Docker network reference	<code>docker compose down -v</code> ; <code>docker network prune</code> , restart clean
<code>seed:demo</code> succeeds but <code>findings total</code> is 0	The <code>data/</code> fixture volume is not mounted	Verify <code>./data:/app/data:ro</code> , rebuild
Test suite reports a <code>beforeAll</code> timeout on a handful of unrelated files	CPU contention transforming ~145 files cold, in parallel, on a constrained machine — a documented flake	Re-run the specific files alone; they pass in isolation
A provider is stuck at <code>FAILED/INVALID_KEY</code>	Wrong key, expired key, or wrong auth scheme	Check Part 6's exact mechanism for that provider
Login works but a page renders "Not available to your role"	Correct behavior — the role lacks the required capability	Check Appendix 13.1

10.10 Maintenance windows

No formal maintenance-window process exists for this prototype; `docker compose restart backend` applies a code or config change without deleting data. A production deployment would need a documented window policy — Part 9, §9.2.

10.11 Release checklist (current state)

1. `git status` clean; correct branch checked out.
2. `npx prisma validate` and `npx prisma migrate diff --exit-code` — zero drift.
3. Backend suite green against real PostgreSQL (`npm test`, serial file execution for real-DB suites).
4. All nine core evaluators green (§11.3).
5. Frontend lint, test, and build green.
6. CI green on the pushed branch.
7. `docs/ai/STATE.yaml` and `docs/ai/HANDOFF.md` updated (internal AI-development-workflow bookkeeping, not part of the shipped product).

10.12 Rollback checklist (current state)

There is no automated rollback mechanism. To roll back a bad deploy on a local/demo Compose stack: `git checkout <last-good-commit>`, `docker compose up --build` again — migrations are additive-only, so rolling the application code back never requires rolling the schema back. A genuine schema rollback is not supported by this project's additive-only migration policy and is not attempted.

10.13 Demo-day operational checklist

1. Rehearse the exact stack the night before, end to end (§8, §10.8).
 2. Confirm the four seeded accounts exist and the password is known.
 3. Rehearse the offline path if venue internet is uncertain.
 4. Have the browser open to the login page at a comfortable zoom level before anyone is watching.
 5. Know in advance which finding/case will be opened — do not hunt for it live.
 6. Turn off notification pop-ups on the presenting machine.
 7. Keep `docker compose -f docker-compose.yml -f docker-compose.offline.yml up -d` ready as a backup command if connectivity fails mid-demonstration.
-

Part 11 — Testing, CI and security assurance

11.1 Four independent verification layers

Unit/integration tests (does this function/route behave correctly) · evaluators (does the real system reproduce hand-authored ground truth end to end) · the browser suite (does it work in a real browser against a real backend) · CI's own hygiene checks (nothing secret or generated committed, schema and migration history intact).

11.2 Unit and integration tests

145 test files across `backend/tests/{unit,integration,middleware}`, run against a real PostgreSQL database — no in-memory fake. Provider tests never touch the network: every provider test injects its own fake `fetch`, so the full suite runs with zero provider keys and zero outbound requests, proven directly (some integration tests explicitly set `ABUSEIPDB_API_KEY: ""/NVD_API_KEY: ""` before `src/config/env` is first required, so behavior does not depend on a developer's own `.env`).

Test count: `docs/TESTING_AND_CI.md` records **3071 passed / 177 skipped** as of Phase 8F (the last phase to change backend code). This dossier reconciled that figure against the current tree by counting test files directly (`find backend/tests -name "*.test.js"`): **145 files**, an exact match to the documented count. No backend source file changed between Phase 8F and this dossier's commit (`c3f8a4b`), so the pass/skip figures are carried forward as **verified at commit `c3f8a4b`, reconfirmed by file-count parity** rather than re-executed in this drafting session — the full suite requires a live PostgreSQL instance this documentation-only session did not stand up. Frontend: Vitest + Testing Library, plus `oxlint`.

11.3 Evaluators

Nine gates, each driving the real production services end to end against a disposable database and comparing against hand-authored ground truth (not mocked expectations), run on every push:

Command	Covers
<code>npm run eval:phase1</code>	Ingestion, dedup, persistence, recurrence
<code>npm run eval:risk</code>	Risk v1 determinism and explainability
<code>npm run eval:phase2</code>	Ownership resolution, IOC enrichment, consistency detection
<code>npm run eval:phase3</code>	Analyst workflow, closure, recurrence reopening
<code>npm run eval:phase4</code>	Notification review, export, delivery tracking
<code>npm run eval:phase5</code>	Framework mappings, guarded AI assistance
<code>npm run eval:phase6.3</code>	ATT&CK catalogue and evidence integrity
<code>npm run eval:vulnerability</code>	CVE association, NVD/KEV/EPSS evidence
<code>npm run eval:phase7</code>	No-key startup, offline operation, AI off

Two additional gates (eval:phase2:mutation, eval:vulnerability:mutation) take minutes rather than seconds and run only on manual workflow_dispatch — not on every push. The six live-provider phases (8B/8D/8E/8F, plus AbuseIPDB/NVD which predate the evaluator/provider split) have no dedicated phase evaluator; each is proven by its own unit/integration/route-authorization/evidence test suite instead, because each is an isolated adapter addition rather than a cross-cutting workflow change.

11.4 Frontend and Chromium browser suite

```
cd frontend; npm run lint; npm test; npm run build; npm run test:e2e
```

9 Playwright spec files (frontend/e2e/*.spec.js) against the real stack, Chromium only, covering the dashboard, findings/upload flow, role-based access, the ATT&CK navigator, Finding-level AI assistance states, responsive breakpoints, motion/reduced-motion behavior, and session handling (401 vs. 403). **Known trap:** reuseExistingServer-style configuration can silently attach to a leftover preview server and produce a false result — always confirm the suite is testing the server just started, on a dedicated port.

11.5 CI pipeline

.github/workflows/ci.yml runs six jobs on every push (a seventh, deep-gates, is manual-dispatch only):

Job	Verifies
hygiene	No committed .env file, no credential-shaped literal, no committed build output/node_modules, no unresolved whitespace/conflict markers
schema	prisma validate; the pinned ATT&CK catalogue checksum; migration count and order match a frozen, reviewed list; migrate deploy succeeds from an empty database; zero drift between schema.prisma and applied migrations
backend	The backend suite against a real, disposable PostgreSQL service container
frontend	Lint, test, build, and a scan of the production JS bundle for provider-key-shaped strings
e2e	Seeds a disposable database through the real REST API, builds the frontend, starts the backend, runs the Chromium suite
evaluators	All nine core evaluators (§11.3) against a disposable database
deep-gates (manual only)	The two mutation/concurrency gates

11.6 Provider no-live-call guarantee

Enforced at three independent layers: (1) every provider unit test injects its own fake transport; (2) every CI job that runs backend code sets provider keys to an explicit empty string rather than leaving them unset/inherited; (3) eval:phase7 replaces global.fetch with a function that throws if called and asserts the call count is exactly zero across the whole gate — a structural proof, not a hope.

11.7 Known flake policy

CPU-contention beforeAll timeout: a handful of integration test files can hit a 10-second hook timeout when the full ~145-file suite transforms cold in parallel on a resource-constrained machine — confirmed non-systemic every time observed; the same files pass cleanly in isolation. **CI real-Postgres concurrency timing:** a real-database concurrency test can occasionally lose a race under shared CI-runner load; policy is to verify the failing test's file was not touched by the change under review, then re-run before treating it as real. Chromium is the only committed browser gate — Firefox and WebKit are not run and are not claimed.

11.8 Security controls (summary; full detail in Part 3 §3.4 and the Security appendix)

Backend-only authorization boundary · structural route census requiring every mounted route to authenticate and enforce a capability · three independent rate-limit buckets · per-provider credential redaction proven by dedicated tests · CI bundle scan for secret-shaped literals · append-only audit logging from the service layer · constant-shape login responses regardless of account existence.

11.9 Current security-assessment limitation

No independent penetration test or third-party security assessment has been performed on this codebase as part of this delivery. This is stated plainly rather than implied to have happened by the presence of other controls. The controls documented in this dossier (route census, redaction tests, rate limiting, audit logging, CI hygiene scanning) are internal, automated, and repository-verifiable; none of them substitutes for an independent, adversarial review. See Part 9, §9.2 and §9.3 for this as a named production-acceptance gap.

Part 12 — User and administrator guide

12.1 Signing in

Open the frontend and sign in with an email and password. Local demo accounts (created by `npm run seed:users`, never present in a real deployment):

Role	Account
Administrator	admin@threatnexus.local
Analyst	analyst@threatnexus.local
Reviewer	reviewer@threatnexus.local
Viewer	viewer@threatnexus.local

A session lasts as long as the issued JWT (JWT_EXPIRES_IN, 24h default). There is no "remember me" or refresh mechanism.

12.2 Dashboard

Every figure carries **value · availability · source · asOf**. Rendering the dashboard makes zero live provider requests. A Finding with no current risk score is counted separately, never folded into the lowest band. Framework mappings render as "analyst-associated framework context," with no coverage percentage and no denominator over any catalogue. A section a role cannot read shows "Not available to your role," never a zero.

12.3 Findings

One row is one (indicator, port, protocol, report type) identity; re-uploading the same host appends an occurrence rather than creating a duplicate row. Invalid filters are rejected by name, never silently ignored. Opening a Finding shows ownership (with confidence labeling — an ASN-based attribution is labeled low-confidence), the Risk v1 explanation table (APPLIED/NOT_AVAILABLE/NOT_APPLICABLE), a per-provider enrichment panel, and — if `AI_ENABLED=true` and a provider is configured — an AI assistance panel.

12.4 Upload (ADMIN, ANALYST)

Uploads a Shadowserver-style Accessible-RDP CSV. The capability check runs before file-parsing middleware, so a denied caller never causes a temp file to be written.

12.5 Triage (ADMIN, ANALYST)

The entry point into the case workflow — deciding what needs a case, then opening one bound to an organization.

12.6 Cases

Everyone can read a case; writing requires `manage:cases`. The analyst who requests a closure can never approve their own request — verified as an actual 403, not a policy statement. A REMEDIATED closure additionally requires a recorded REMEDIATED organization response.

12.7 Framework mapping

An ATT&CK mapping justified only by exposure or risk score is refused server-side. A mapping cites a verbatim stored quote and carries its own confidence value, separate from any other score.

12.8 AI assistance

Request a summary or explanation draft (ADMIN/ANALYST); accept or reject it if the role holds `review:ai-suggestions` (ADMIN/REVIEWER). A draft is never presented as a finding fact.

12.9 Notifications (ADMIN/ANALYST draft; ADMIN/REVIEWER approve — never VIEWER)

Draft from case evidence, revise (each edit is a new immutable revision, invalidating any prior approval), submit for review, export as `.eml` once approved, record delivery separately.

12.10 Denied and restricted states

Denied (403) — the role lacks the capability; shown inline, never as a silent disappearance. **Restricted** — a dashboard section the role cannot read at all; shown as "Not available to your role," never a zero. Neither is a bug to work around.

12.11 Provider configuration (ADMIN)

Environment-variable only — there is no in-app provider-configuration screen. Export the key in the shell that runs `docker compose up`, then restart the backend. No key requires a migration or code change.

12.12 Admin limitations (stated plainly)

No in-app user management — accounts come from the seed script or a direct database change. No in-app audit-log viewer — query the `AuditLog` table directly. No token revocation — a role change or account disablement takes effect only when the existing JWT expires. No in-app provider-key configuration screen. No production deployment target exists for this project.

Part 13 — Appendices

13.1 Full capability matrix

Capability	ADMIN	ANALYST	REVIEWER	VIEWER
read:dashboard	✓	✓	✓	✓
read:findings	✓	✓	✓	✓
read:cases	✓	✓	✓	✓
ingest:reports	✓	✓		
triage:findings	✓	✓		
manage:cases	✓	✓		
override:finding-ownership	✓	✓		
trigger:finding-enrichment	✓	✓		
recalculate:finding-risk	✓	✓		
manage:finding-vulnerabilities	✓	✓		
trigger:vulnerability-enrichment	✓	✓		
read:notifications	✓	✓	✓	
manage:notifications	✓	✓		
export:notifications	✓	✓		
record:notification-delivery	✓	✓		
manage:framework-mappings	✓	✓		
read:ai-mapping-suggestions	✓	✓	✓	
request:ai-mapping-suggestions	✓	✓		
decide:ai-mapping-suggestions	✓	✓		
read:ai-finding-suggestions	✓	✓	✓	
request:ai-finding-suggestions	✓	✓		
review:notifications	✓		✓	
review:ai-suggestions	✓		✓	
review:case-closure	✓		✓	
manage:users (granted, unrouted — no feature consumes it yet)	✓			
manage:system	✓			
delete:records	✓			
manage:ownership-mappings	✓			

Capability	ADMIN	ANALYST	REVIEWER	VIEWER
execute:enrichment-batch	✓			
execute:vulnerability-enrichment-batch	✓			
override:closure-self-approval	✓			
override:notification-self-approval	✓			

An unrecognized role in a JWT resolves to **zero capabilities**, never to VIEWER's read-only set.

13.2 Current API route catalogue

Generated directly from backend/src/routes/*.js and backend/src/lib/roles.js at commit c3f8a4b — not from docs/API_CONTRACT_PHASE0.md alone, which documents only the Phase 0 surface and is explicitly labeled partial below (§13.2.1). All routes require Authorization: Bearer <JWT> unless marked "none"/"authenticated only."

Method	Path	Capability	Purpose
POST	/api/auth/register	none	Register a new user (always VIEWER, regardless of requested role)
POST	/api/auth/login	none	Authenticate and issue a JWT
GET	/api/profile	authenticated only	Return current user identity and capability list
GET	/api/threats	read:findings	List legacy Threat records (Phase 0 surface, still present)
GET	/api/threats/search	read:findings	Search legacy Threat records
POST	/api/threats/upload	ingest:reports	Generic CSV importer (legacy, not the Finding pipeline)
PATCH	/api/threats/:id/status	triage:findings	Update legacy threat status
DELETE	/api/threats/:id	delete:records	Delete a legacy threat record

Method	Path	Capability	Purpose
POST	/api/reports/upload	ingest:reports	Upload an Accessible-RDP report (the Phase 1 pipeline)
GET	/api/dashboard/overview	read:dashboard	The one truthful, bounded operational snapshot
GET	/api/dashboard/stats	read:dashboard	Legacy dashboard stats
GET	/api/dashboard/charts	read:dashboard	Legacy dashboard chart data
GET	/api/cases	read:cases	List cases
GET	/api/cases/:id	read:cases	Get a case
GET	/api/cases/:id/workflow	read:cases	Get case workflow state
POST	/api/cases	manage:cases	Create a case
PUT	/api/cases/:id	manage:cases	Update a case
DELETE	/api/cases/:id	manage:cases	Tombstone endpoint — always 409, deletes nothing
POST	/api/cases/:id/findings	manage:cases	Link a Finding to a case
POST	/api/cases/:id/findings/:findingId/unlink	manage:cases	Unlink a Finding
POST	/api/cases/:id/state	manage:cases	Change case state
POST	/api/cases/:id/responses	manage:cases	Record an organization response
POST	/api/cases/:id/closure-requests	manage:cases	Request case closure
POST	/api/cases/:id/reopen	manage:cases	Reopen a case
POST	/api/cases/:id/closure-requests/:requestId/approve	review:case-closure	Approve a closure request
POST	/api/cases/:id/closure-requests/:requestId/reject	review:case-closure	Reject a closure request

Method	Path	Capability	Purpose
GET	/api/cases/:id/framework-mappings	read:cases	List active framework mappings
GET	/api/cases/:id/framework-mappings/history	read:cases	List mapping history
POST	/api/cases/:id/framework-mappings	manage:framework-mappings	Create/reactivate a mapping
POST	/api/cases/:id/framework-mappings/:mappingId/remove	manage:framework-mappings	Remove a mapping
GET	/api/cases/:id/framework-mappings/no-applicable	read:cases	List "no applicable" assertions
POST	/api/cases/:id/framework-mappings/no-applicable	manage:framework-mappings	Assert no applicable mapping
POST	/api/cases/:id/framework-mappings/no-applicable/:assertionId/withdraw	manage:framework-mappings	Withdraw an assertion
GET	/api/cases/:id/ai/mapping-suggestions	read:ai-mapping-suggestions	List AI mapping suggestions
POST	/api/cases/:id/ai/mapping-suggestions	request:ai-mapping-suggestions	Request AI mapping suggestions
POST	/api/cases/:id/ai/mapping-suggestions/:suggestionId/approve	decide:ai-mapping-suggestions	Approve a suggestion
POST	/api/cases/:id/ai/mapping-suggestions/:suggestionId/reject	decide:ai-mapping-suggestions	Reject a suggestion
GET	/api/notifications	read:notifications	List notifications
GET	/api/notifications/:id	read:notifications	Get a notification
GET	/api/notifications/:id/history	read:notifications	Get revision history
GET	/api/notifications/:id/export	export:notifications	Download the approved .eml artifact
POST	/api/notifications	manage:notifications	Draft a notification from a case
PUT	/api/notifications/:id	manage:notifications	Edit — produces a new revision
POST	/api/notifications/:id/submit	manage:notifications	Submit for review

Method	Path	Capability	Purpose
POST	/api/notifications/:id/approve	review:notifications	Approve
POST	/api/notifications/:id/reject	review:notifications	Reject
POST	/api/notifications/:id/deliveries	record:notification-delivery	Record a delivery outcome
POST	/api/notifications/:id/responses	manage:cases	Record an org response via the notification
DELETE	/api/notifications/:id	manage:notifications	Tombstone endpoint — always 409
GET	/api/organizations/options	manage:cases	Bounded organization options list
GET / POST / PUT / DELETE	/api/organizations[:id]	manage:system	Organization CRUD (ADMIN only)
GET	/api/ownership/coverage	read:findings	Ownership coverage report
GET	/api/ownership/mappings	read:findings	List asset ownership mappings
POST	/api/ownership/mappings	manage:ownership-mappings	Create an asset mapping
PATCH	/api/ownership/mappings/:id	manage:ownership-mappings	Update a mapping
POST	/api/ownership/mappings/:id/disable	manage:ownership-mappings	Disable a mapping
POST	/api/ownership/mappings/:id/re-resolve	manage:ownership-mappings	Retry ownership re-resolution
POST	/api/ownership/consistency-check	manage:ownership-mappings	Report ownership defects
GET	/api/findings	read:findings	List Findings
GET	/api/findings/:id	read:findings	Get a Finding
GET	/api/findings/:id/ownership	read:findings	Get Finding ownership
PUT / DELETE	/api/findings/:id/ownership/override	override:finding-ownership	Set/clear an ownership override

Method	Path	Capability	Purpose
GET	/api/findings/:id/enrichments	read:findings	List AbuseIPDB (IOC) enrichment results
POST	/api/findings/:id/enrichment	trigger:finding-enrichment	Schedule IOC enrichment
GET / POST	/api/findings/:id/enrichment/censys	read:findings / trigger:finding-enrichment	Read / trigger Censys
GET / POST	/api/findings/:id/enrichment/greynoise	read:findings / trigger:finding-enrichment	Read / trigger GreyNoise
GET / POST	/api/findings/:id/enrichment/shodan	read:findings / trigger:finding-enrichment	Read / trigger Shodan
GET / POST	/api/findings/:id/enrichment/netlas	read:findings / trigger:finding-enrichment	Read / trigger Netlas
GET	/api/findings/:id/risk	read:findings	Get the current Risk v1 score
POST	/api/findings/:id/risk/recalculate	recalculate:finding-risk	Recalculate risk
GET	/api/findings/:id/vulnerabilities	read:findings	List CVE associations
POST	/api/findings/:id/vulnerabilities	manage:finding-vulnerabilities	Attach a CVE
POST	/api/findings/:id/vulnerabilities/:cveId/remove	manage:finding-vulnerabilities	Remove a CVE association
GET / PUT	/api/findings/:id/triage	read:findings / triage:findings	Read / set triage decision
GET	/api/findings/:id/ai-suggestions	read:ai-finding-suggestions	List Finding AI drafts
POST	/api/findings/:id/ai-suggestions	request:ai-finding-suggestions	Request a draft
POST	/api/findings/:id/ai-suggestions/:suggestionId/accept	review:ai-suggestions	Accept a draft
POST	/api/findings/:id/ai-suggestions/:suggestionId/reject	review:ai-suggestions	Reject a draft

Method	Path	Capability	Purpose
POST	/api/enrichment/batches/run	execute:enrichment-batch	Run the bounded IOC enrichment batch worker
POST	/api/vulnerabilities/:cveId/enrichment	trigger:vulnerability-enrichment	Schedule CVE vulnerability enrichment
POST	/api/vulnerability-enrichment/batches/run	execute:vulnerability-enrichment-batch	Run the vulnerability batch worker
GET	/api/attack/catalogue	read:cases	Get the pinned ATT&CK catalogue
GET	/api/attack/navigator	read:cases	Get the navigator with mapping counts (no coverage %)
GET	/api/ai/config	read:cases	Get AI assistance availability (shared by both AI surfaces)

13.2.1 Phase 0 contract status

`docs/API_CONTRACT_PHASE0.md` documents the Phase 0 surface only (`/auth`, `/profile`, the legacy `/threats` group, and the flat, pre-workflow `/cases//notifications//organizations` CRUD groups as they existed at that phase) and remains accurate for what it covers. It is explicitly a **partial** reference — it predates Phases 1 through 8F and does not describe the Finding/Case/Notification workflow surface documented above. The table in §13.2 is the current, complete reconciliation; no single maintained document previously covered Phases 1–8F's routes in one place before this dossier.

13.3 Environment-variable catalogue

See Part 8, §8.4 for the categorized table. The authoritative source is always `backend/.env.example` at the current commit — this dossier reproduces it for reference, not as a second source of truth.

13.4 Provider comparison

Provider	Domain	Auth mechanism	Key required?	Own table
AbuseIPDB	IOC reputation	API key header	Optional	IocEnrichment (queued)

Provider	Domain	Auth mechanism	Key required?	Own table
NVD	Vulnerability metadata	Optional API key	Optional (keyless = lower rate limit)	VulnerabilityProviderResult
CISA KEV	Vulnerability metadata	None	No	VulnerabilityProviderResult
FIRST EPSS	Vulnerability metadata	None	No	VulnerabilityProviderResult
Censys	Exposure / attack surface	Bearer PAT	Optional	CensysEnrichment
GreyNoise	Reputation / noise	key header	Optional	GreyNoiseEnrichment
Shodan	Exposure / attack surface	key query param	Optional	ShodanEnrichment
Netlas	Exposure / attack surface	Bearer header	Optional	NetlasEnrichment

13.5 Status and error vocabulary

Vocabulary	Values
Provider config status	CONFIGURED, NOT_CONFIGURED, CONFIGURED_WITH_KEY, KEYLESS_PUBLIC_RATE_LIMIT, NO_KEY_REQUIRED, MOCK_PROVIDER, ENABLED, DISABLED
Provider freshness	FRESH, STALE, NO_SUCCESSFUL_LOOKUP_RECORDED
Provider error codes	PROVIDER_RATE_LIMITED, PROVIDER_INVALID_KEY, PROVIDER_TIMEOUT, PROVIDER_UNAVAILABLE, PROVIDER_UNREACHABLE, PROVIDER_MALFORMED_RESPONSE, PROVIDER_REJECTED, PROVIDER_DISABLED/ENRICHMENT_DISABLED, UNSUPPORTED_IDENTIFIER/UNSUPPORTED_INDICATOR, CATALOG_UNAVAILABLE
Risk factor applicability	APPLIED, NOT_AVAILABLE, NOT_APPLICABLE
Dashboard figure availability	a real value, RESTRICTED, UNAVAILABLE
AI suggestion status	DRAFT, ACCEPTED, REJECTED, EXPIRED
Audit outcome	SUCCESS, FAILURE, DENIED
Auth error	401 Authentication required. / 403 Forbidden. (fixed bodies; the missing capability is never named to the caller, only to the audit row)

13.6 Requirements traceability matrix (representative sample)

Requirement	Implementing module	Verifying test/evaluator
TNX-FR-003/004/005 (dedup/persistence/recurrence)	backend/src/services/ingestion/	eval:phase1
TNX-FR-010 (Risk v1)	backend/src/services/risk/	eval:risk
TNX-FR-006 (ownership resolution)	backend/src/services/ownership/	eval:phase2, eval:phase2:mutation
TNX-FR-011–015 (case workflow, closure)	backend/src/services/workflow/	eval:phase3
TNX-FR-021–026 (notification lifecycle)	backend/src/services/notification/	eval:phase4
TNX-FR-016/017 (framework mapping)	backend/src/services/mapping/	eval:phase5, eval:phase6.3
TNX-FR-018–020, TNX-AI-* (AI assistance)	backend/src/services/ai/, backend/src/services/aiAssist/	eval:phase5; aiSafetyBoundaries.test.js; findingAiPromptInjection.test.js
TNX-NFR-001/002/008 (AI/provider off, no live calls)	provider registries + eval harness	eval:phase7
TNX-SEC-002 (every route authenticated + capability-gated)	backend/src/app.js router mounts	phase7RouteCensus.test.js
TNX-SEC-005/006 (separation of duties)	backend/src/services/workflow/, notification/	eval:phase3, eval:phase4, seed:demo (live 403 during seeding)
TNX-NFR-005 (additive-only migrations)	backend/prisma/migrations/	CI schema job's frozen migration-history check

A complete row-per-requirement matrix follows the same pattern and is maintained alongside the SRS in Part 3; the sample above demonstrates the traceability method rather than repeating all 60+ requirement rows a second time.

13.7 Production-readiness checklist (starting point, not yet completed)

- ☐ Independent security assessment completed and findings accepted or closed
- ☐ TLS terminated at a reverse proxy with a real domain and certificate
- ☐ Secret manager in place; no credential in a process environment variable set by hand
- ☐ Shared rate-limit store deployed; in-process limiting retired
- ☐ Managed or hardened PostgreSQL with replication and monitored backups
- ☐ At least one successful, timed restore drill from a real backup
- ☐ RPO/RTO targets set and accepted by a named stakeholder
- ☐ Centralized logging, metrics, and alerting operational
- ☐ Release and rollback procedures documented and rehearsed at least once
- ☐ Token revocation or SSO integration implemented
- ☐ Dependency/vulnerability scanning wired into CI

- [] Incident-response procedure documented
- [] Production acceptance sign-off recorded by a named approver

13.8 Known limitations (consolidated)

Single report type (Accessible RDP) · manual CSV ingestion only, no live Shadowserver feed · no live AI provider · **Finding closure has no production write path** (the recurrence engine is real and evaluator-proven; nothing in the UI/API currently produces the closed state it reopens from) · Chromium-only browser gate · only ATT&CK is catalogue-verified, NIST CSF/CIS are shape-checked only · the ATT&CK lexical guard is a backstop, not comprehension · ASN-tier ownership cannot be re-resolved later (ASN is not persisted) · no production deployment story · in-process rate limiting · synthetic data only, by design · no in-app user management · no in-app audit-log viewer · no token revocation · no independent security assessment performed on this delivery.

13.9 Document references

README.md · docs/PROJECT_PLAYBOOK.md · docs/ARCHITECTURE.md · docs/PROVIDER_GUIDE.md · docs/AI_GOVERNANCE.md · docs/DELIVERY.md · docs/DEPLOYMENT.md · docs/OPERATIONS_RUNBOOK.md · docs/TESTING_AND_CI.md · docs/USER_GUIDE.md · docs/ADMIN_GUIDE.md · docs/DEMO_SCRIPT.md · docs/DEMO_RUNBOOK.md · docs/API_CONTRACT_PHASE0.md · docs/ai/SECURITY.md · ../ThreatNeXus-Planning/planning/DECISIONS.md (D-001, the Node/Express/Prisma stack decision) · docs/TEAM_STUDY_GUIDE.md (a separate, internal team-learning artifact — consulted read-only as supplementary context, never merged into this dossier's content).

End of dossier. See docs/delivery/DOSSIER_BUILD_NOTES.md for source reconciliation detail, design decisions, validation performed, and regeneration instructions.